# Transformer 块

## The Transformer Block

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

# 专业术语翻译对照表

| 英文原词 | 中文译名 | 缩写／符号 |
| --- | --- | --- |
| Transformer Block | Transformer 块 | |
| Normalization | 归一化 | |
| Attention | 注意力机制 | Attention |
| Matrix Multiplication | 矩阵乘法 | |
| Incremental Decoding | 增量解码 | |
| Feed-forward Network | 前馈网络 | FFN |
| Output Projection | 输出投影 | |
| Residual Stream | 残差流 | |
| Head Dimension | 头维度 | |
| Feed-forward | 前馈 | |
| Batch Size | 批大小 | |
| Activation | 激活 | |
| Embeddings | 嵌入向量 | |
| LayerNorm | 层归一化 | LayerNorm |
| Training | 训练 | |
| KV Cache | 键值缓存 | KV Cache |
| Prefill | 预填充 | |
| RMSNorm | 均方根归一化 | RMSNorm |
| Serving | 推理服务 | |
| Kernels | 内核 | |
| Tensor | 张量 | |
| Models | 模型 | |
| SwiGLU | SwiGLU 门控激活 | |
| Query | 查询 | Q |
| Value | 值 | V |
| Model | 模型 | |
| Key | 键 | K |
| Self-attention | 自注意力 | |
| Representation | 表示 | |
| Transformer | Transformer 模型 | Transformer |
| Pre-norm | 前置归一化 | |
| Decoder | 解码器 | |

| 英文原词 | 中文译名 | 缩写／符号 |
| --- | --- | --- |
| Vectors | 向量 | |
| Gating | 门控 | |
| Token | 词元 | |
| Residual Connections | 残差连接 | |
| Mental Model | 理解框架 | |
| Heads | 头 | |
| Head Dimensions | 头维度 | |
| Query Heads | 查询头 | |
| Value Heads | 值头 | |
| KV Heads | 键值头 | |
| Residual Connection | 残差连接 | |
| Cross-attention | 交叉注意力 | |
| Encoder | 编码器 | |
| Layer Normalization | 层归一化 | LayerNorm |
| Residual Path | 残差路径 | |
| Output Head | 输出头 | |
| Tensor Shape | 张量形状 | |
| Core | 核心 | |
| Vector | 向量 | |
| Evidence | 证据 | |
| Scaling | 缩放 | |
| Scaled Dot-product Attention | 缩放点积注意力 | SDPA |
| Learned Projections | 学习得到的投影 | |
| Attention Weights | 注意力权重 | |
| Matrix | 矩阵 | |
| Rotary Position Embedding | 旋转位置嵌入 | RoPE |
| Token Embedding | 词元嵌入 | |
| Latent Representation | 潜在表示 | |
| Key/Value Heads | 键值头 | KV Head |
| Projection Matrices | 投影矩阵 | |
| Hidden State | 隐藏状态 | |
| Hidden Representation | 隐藏表示 | |
| Projection Matrix | 投影矩阵 | |
| Activations | 激活 | |

| 英文原词 | 中文译名 | 缩写／符号 |
| --- | --- | --- |
| Baseline | 基线 | |
| Rounding | 舍入 | |
| Decoder-only LLM | 仅解码器大语言模型 | |
| Queries | 查询 | |
| Values | 值 | |
| Keys | 键 | |
| Dot-product Attention | 点积注意力 | |
| Additive Mask | 加性掩码 | |
| Mixture-of-Experts | 混合专家 | MoE |
| Sparse Attention | 稀疏注意力 | |
| Experts | 专家 | |
| Expert | 专家 | |
| Router | 路由器 | |
| Global Attention | 全局注意力 | |
| Post-norm | 后置归一化 | |
| Causal Mask | 因果掩码 | |
| Tokens | 词元 | |
| Batching | 批处理 | |
| Matrices | 矩阵 | |
| Kernel | 内核 | |
| Linear Projections | 线性投影 | |
| Precision | 精度 | |
| Recomputation | 重计算 | |
| Fused Kernels | 融合内核 | |
| Forward Pass | 前向传播 | |
| Inference | 推理 | |
| Gradient | 梯度 | |
| Dropout | 随机失活 | |
| Logit | 逻辑值（归一化前的分数） | |
| Exact Attention | 精确注意力 | |
| Token Embeddings | 词元嵌入 | |
| Local Attention | 局部注意力 | |
| Routing | 路由 | |
| Attention Head | 注意力头 | |

| 英文原词 | 中文译名 | 缩写／符号 |
| --- | --- | --- |
| FlashAttention | FlashAttention（方法名保留） | |

# 目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。


封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

Transformer 块（Transformer Block） · 第 8 页 ［原文 1］

- 方向 · 第 10 页 ［原文 2］

1 - Transformer 块（Transformer Block） 是什么 · 第 12 页 ［原文 3］

2 - 残差流（Residual Stream） · 第 14 页 ［原文 4］

3 - 穿过块的形状 · 第 16 页 ［原文 5］

4 - 原 2017 年 LN 后区块 · 第 18 页 ［原文 6］

5 - 现代 PRE-LN 解码器（Decoder） 块 · 第 20 页 ［原文 7］

6 - 为什么残留添加很重要 · 第 22 页 ［原文 8］

7 - 层归一化（LayerNorm） 力学 · 第 24 页 ［原文 9］

8 - 均方根归一化（RMSNorm） 力学 · 第 26 页 ［原文 10］

9 - 工作正常示例 · 第 28 页 ［原文 11］

10 - LN 前 VS LN 后 · 第 30 页 ［原文 12］

11 - 注意力机制（Attention） 分支机构 · 第 32 页 ［原文 13］

12 - 绳索进入的位置 · 第 34 页 ［原文 14］

13 - 块内的 MHA、GQA、MQA 和 MLA · 第 36 页 ［原文 15］

14 - 输出投影（Output Projection） 关闭 注意力机制（Attention） 分支机构 · 第 38 页 ［原文 16］

15 - MLP 是位置明智的 · 第 40 页 ［原文 17］

16 - 原始 RELU 前馈网络（Feed-forward Network） · 第 42 页 ［原文 18］

17 - 从 GELU 到门控 MLPS · 第 44 页 ［原文 19］

18 - SwiGLU 门控激活（SwiGLU） 力学 · 第 46 页 ［原文 20］

19 - 为什么门控 MLPS 使用不同的宽度 · 第 48 页 ［原文 21］

20 - 工作参数示例 · 第 50 页 ［原文 22］

21 - 加工形状追踪 · 第 52 页 ［原文 23］

22 - 串行与并行剩余块 · 第 54 页 ［原文 24］

23 - 编码器（Encoder） 和 解码器（Decoder） 块系列 · 第 56 页 ［原文 25］

24 - 因果屏蔽位于 注意力机制（Attention） 内部 · 第 58 页 ［原文 26］

25 - MOE 取代 MLP 分支 · 第 60 页 ［原文 27］

26 - QK-NORM 不是块 归一化（Normalization） · 第 62 页 ［原文 28］

27 - GEMMA 2：一个有用的反例 · 第 64 页 ［原文 29］

28 - 训练（Training）、预填充（Prefill） 和解码 · 第 66 页 ［原文 30］

29 - 键值缓存（KV Cache） 属于 注意力机制（Attention） 状态 · 第 68 页 ［原文 31］

30 - 块级参数核算 · 第 70 页 ［原文 32］

31 - 计算直觉：注意力机制（Attention） VS MLP · 第 72 页 ［原文 33］
32 - 激活（Activation） 内存和 重计算（Recomputation） · 第 74 页 ［原文 34］
33 - 随机失活（Dropout）、偏见和其他小选择 · 第 76 页 ［原文 35］
34 - 透明的 Pytorch 块 · 第 78 页 ［原文 36］
35 - 常见的实施错误 · 第 80 页 ［原文 37］
36 - 常见的误解 · 第 82 页 ［原文 38］
37 - 完整的 理解框架（Mental Model） · 第 84 页 ［原文 39］
38 - 紧凑架构比较 · 第 86 页 ［原文 40］
39 - 参考文献和扩展阅读 · 第 88 页 ［原文 41］

# Transformer 块（Transformer Block）

对应原文第 1 页；点击查看相邻原文对照页

了解 Transformer 块（Transformer Block） 从残差流和 注意力机制（Attention） 到 MLP、归一化（Normalization） 和现代 LLM 模块 面向 AI 工程师、开发人员和认真学习者的简短技术手册 手册 09 - 了解 AI 系列

# Understanding the Transformer Block

From Residual Streams and Attention to MLPs, Normalization, and Modern LLM Blocks

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 09 - Understanding AI Series

# - 方向

对应原文第 2 页；点击查看相邻原文对照页

- 方向 本手册将阐明的内容 Transformer 块（Transformer Block） 在图表中看起来很简单：归一化（Normalization）、注意力机制（Attention）、前馈网络（Feed-forward Network） 和两个剩余添加项。困难在于现代语言 模型（Models） 几乎改变了该骨架周围的每个细节。有些在子层之前进行标准化，有些在子层之后进行标准化，有些则两者都进行。

前馈（Feed-forward） 分支可以使用 ReLU、GELU、SwiGLU 门控激活（SwiGLU）、GeGLU 或稀疏 MoE。 注意力机制（Attention） 在解码期间可以使用 MHA、GQA、MQA、MLA、RoPE、QK-Norm、本地窗口或 键值缓存（KV Cache）。

本手册将稳定结构与特定于体系结构的选择分开。到最后，您应该能够解释：

- 哪些信息流过残差流（Residual Stream），以及为什么每个子层必须返回模型（Model） 宽度；

- Post-LN 和 Pre-LN 在代数上有何不同，而不仅仅是图表上的区别；

- 层归一化（LayerNorm） 和均方根归一化（RMSNorm） 实际计算什么；

- 注意力机制（Attention） 在块内开始和结束的位置，包括 RoPE、GQA 和 输出投影（Output Projection）；

- 为什么MLP 是位置明智的，而整个块却不是；

- SwiGLU 门控激活（SwiGLU） 如何更改 前馈（Feed-forward） 计算和参数计数；

- MoE 如何在不改变周围残差拓扑的情况下替换密集的 MLP 分支；

- 训练（Training）、提示符预填充（Prefill） 和增量解码（Incremental Decoding） 之间有何变化；

- 哪些详细信息足够通用以进行推理，哪些必须从模型（Model） 配置中读取。

先决条件 对张量、矩阵乘法（Matrix Multiplication）、注意力机制（Attention） 和神经网络层有基本的了解就足够了。

早期关于 注意力机制（Attention）、RoPE、键值缓存（KV Cache）、MHA/MQA/GQA/MLA、MoE 和 嵌入向量（Embeddings） 的手册是有用的上下文，但这里的块是根据第一原理重建的。范围 这是一本关于 Transformer 块（Transformer Block） 内部计算以及块如何堆叠的手册。它只涉及 内核（Kernels）、分布式执行、激活（Activation） 检查点和 推理服务（Serving） 仅在它们澄清块行为的情况下。这些系统主题值得单独处理。 B = 批大小（Batch Size）;T = 序列长度;d = 模型（Model）/剩余宽度;Hq = 查询（Query）-头数;Hkv = 键（Key）/值（Value）-头数;dh = 头维度（Head Dimension）;m = 前馈（Feed-forward） 中间宽度。除非另有说明，张量（Tensor） 形状是逻辑形状而不是所需的物理内存布局。

研究锚点：[1]、[3]、[4]、[5]、[6]、[7]、[8]、[10]、[16]、[17] 了解 Transformer 块（Transformer Block）

# - ORIENTATION

## What this handbook will make clear

A Transformer block looks simple in diagrams: normalization, attention, a feed-forward network, and two residual additions. The difficulty is that modern language models have changed almost every detail around that skeleton. Some normalize before a sublayer, some after it, some do both. The feed-forward branch may use ReLU, GELU, SwiGLU, GeGLU, or a sparse MoE. Attention may use MHA, GQA, MQA, MLA, RoPE, QK-Norm, local windows, or a KV cache during decoding.

This handbook separates the stable structure from architecture-specific choices. By the end, you should be able to explain:

- what information flows through the residual stream and why every sublayer must return to the model width;
- how Post-LN and Pre-LN differ algebraically, not just diagrammatically;
- what LayerNorm and RMSNorm actually compute;
- where attention begins and ends inside a block, including RoPE, GQA, and the output projection;
- why the MLP is position-wise even though the full block is not;
- how SwiGLU changes the feed-forward computation and parameter count;
- how MoE replaces the dense MLP branch without changing the surrounding residual topology;
- what changes between training, prompt prefill, and incremental decoding;
- which details are universal enough to reason about and which must be read from a model configuration.

## Prerequisites

Basic comfort with tensors, matrix multiplication, attention, and neural-network layers is enough. The earlier handbooks on Attention, RoPE, KV Cache, MHA/MQA/GQA/MLA, MoE, and Embeddings are useful context, but the block is rebuilt here from first principles. **Scope**

This is a handbook about the computation inside one Transformer block and how blocks are stacked. It touches kernels, distributed execution, activation checkpointing, and serving only where they clarify block behavior. Those systems topics deserve separate treatment. **Notation used throughout**

$B$ = batch size; $T$ = sequence length; $d$ = model/residual width; $H_q$ = query-head count; $H_{kv}$ = key/value-head count; $d_h$ = head dimension; $m$ = feed-forward intermediate width. Unless stated otherwise, tensor shapes are logical shapes rather than a required physical memory layout.

*Research anchors: [1], [3], [4], [5], [6], [7], [8], [10], [16], [17]*

# 1 - Transformer 块（Transformer Block） 是什么

对应原文第 3 页；点击查看相邻原文对照页

1 - Transformer 块（Transformer Block） 是什么 重复残差计算 Transformer 模型（Transformer） 是通过堆叠重复更新隐藏 向量（Vectors） 序列的块来构建的。

对于仅 解码器（Decoder） 的语言 模型（Model），让

是进入区块ℓ的剩余表示（Representation）。常见的现代块有两个主要的可训练子层：自注意力（Self-attention） 和 前馈网络（Feed-forward Network） (MLP)。在 前置归一化（Pre-norm） 形式中，

X (ℓ+1) =U + MLP(Norm2(U ))。

注意力机制（Attention） 分支可以跨 词元（Token） 位置混合信息。 MLP 独立作用于每个位置，但会变换其特征维度。剩余添加使两个子层保持连接到共享维度流。

X (ℓ) Norm + 注意力机制（Attention） 残差加法 Norm + MLP 残差加法

重要。“Transformer 块（Transformer Block）”不是一种精确的实现。持久的骨架是围绕 注意力机制（Attention） 和特征混合子层的重复残余更新。 归一化（Normalization） 放置、注意力机制（Attention） 类型、激活（Activation）、门控（Gating） 甚至分支是否串行都可能有所不同。

研究锚点：[1]、[4]、[8]、[10] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 1 - WHAT A TRANSFORMER BLOCK IS

## A repeated residual computation

A Transformer is built by stacking blocks that repeatedly update a sequence of hidden vectors. For a decoder-only language model, let

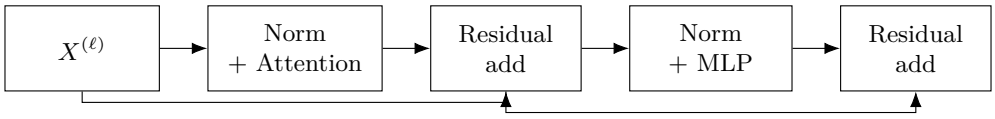
$$X^{(\ell)} \in \mathbb{R}^{B\times T\times d}$$

be the residual representation entering block $\ell$. A common modern block has two main trainable sublayers: self-attention and a feed-forward network (MLP). In a Pre-Norm form,

$$U = X^{(\ell)} + \mathrm{Attn}(\mathrm{Norm}_1(X^{(\ell)})),$$

$$X^{(\ell+1)} = U + \mathrm{MLP}(\mathrm{Norm}_2(U)).$$

The attention branch can mix information across token positions. The MLP acts independently on each position but transforms its feature dimensions. Residual additions keep both sublayers connected to a shared $d$-dimensional stream.




**Important.** "Transformer block" is not one exact implementation. The durable skeleton is repeated residual updates around attention and feature-mixing sublayers. Normalization placement, attention type, activation, gating, and even whether the branches are serial can vary.

*Research anchors: [1], [4], [8], [10]*

# 2 - 残差流（Residual Stream）

对应原文第 4 页；点击查看相邻原文对照页

2 - 残差流（Residual Stream） 每个块读取和更新的状态 短语 残差流（Residual Stream） 是 残差连接（Residual Connections） 通过堆栈携带的 d 维 向量（Vectors） 序列的有用名称。如果子层计算 updateF (x)，则残差规则为

身份路径意味着旧的 表示（Representation） 仍然直接可用，而子层提供学习的修改。这个想法早于变形金刚：引入残差网络是为了通过学习相对于输入的残差函数，使非常深的网络更容易优化。 [2]

对于 Transformer 块（Transformer Block）、注意力机制（Attention） 和 MLP 都将更新写回到相同的 width-d 表示（Representation） 中。该要求解释了一个看起来任意的架构不变量：即使当 注意力机制（Attention） 暂时分裂为 头（Heads） 或 MLP 扩展到更宽的隐藏维度时，分支也必须在剩余添加之前投影回 tod。

理解框架（Mental Model）。块不会将 表示（Representation） 替换为“注意力机制（Attention） 输出”，然后替换为“MLP 输出”。每个分支建议对共享残差 表示（Representation） 进行更新。

研究锚点：[1]、[2] 了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 2 - THE RESIDUAL STREAM

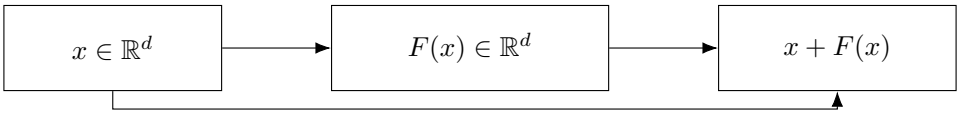
## The state that every block reads and updates

The phrase *residual stream* is a useful name for the sequence of $d$-dimensional vectors carried through the stack by residual connections. If a sublayer computes an update $F(x)$, the residual rule is

$$y = x + F(x).$$

The identity path means the old representation remains directly available while the sublayer contributes a learned modification. This idea predates Transformers: residual networks were introduced to make very deep networks easier to optimize by learning residual functions relative to their inputs. [2]

For a Transformer block, attention and the MLP both write updates back into the same width-$d$ representation. That requirement explains an architectural invariant that can otherwise look arbitrary: even when attention temporarily splits into heads or an MLP expands to a much wider hidden dimension, the branch must project back to $d$ before the residual addition.




**Mental model.** A block does not replace the representation with "the attention output" and then "the MLP output." Each branch proposes an update to a shared residual representation.

*Research anchors: [1], [2]*

# 3 - 穿过块的形状

对应原文第 5 页；点击查看相邻原文对照页

3 - 通过块的形状 在跟踪方程之前跟踪尺寸 对于具有剩余宽度d 的 解码器（Decoder） 块，

归一化（Normalization） 保留此形状。 注意力机制（Attention） 将最后一个维度投影到 查询（Query）、键（Key） 和 值头（Value Heads） 中。与Hq 查询头（Query Heads），Hkv 键值头（KV Heads），和头尺寸dh，

在 注意力机制（Attention） 之后，头输出被组合并投影回

然后，MLP 将每个 词元（Token） 表示（Representation） 扩展为中间宽度 m 并投影回来：

阶段逻辑形态有哪些变化？

范数 B × T × d 统计/尺度，而非形状 Q B × Hq × T × dh 拆分为 查询头（Query Heads） K,V B × Hkv × T × dh 键值头（KV Heads） 可能与 Q 不同 注意力机制（Attention） 输出 B × T × d 头（Heads） 重组/投影 MLP 隐藏 B × T × m 特征扩展 块输出 B × T × d 返回残差流（Residual Stream）

重要。 MLP widthm 和 注意力机制（Attention） 头维度（Head Dimensions） 是临时内部宽度。

余数加法需要分支输出返回tod。

研究锚点：[1]、[11]、[20] 了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 3 - SHAPES THROUGH THE BLOCK

### Track the dimensions before tracking the equations

For a decoder block with residual width $d$,

$$X \in \mathbb{R}^{B\times T\times d}.$$

Normalization preserves this shape. Attention projects the last dimension into query, key, and value heads. With $H_q$ query heads, $H_{kv}$ KV heads, and head dimension $d_h$,

$$Q \in \mathbb{R}^{B\times H_q\times T\times d_h}, \qquad K, V \in \mathbb{R}^{B\times H_{kv}\times T\times d_h}.$$

After attention, head outputs are combined and projected back to

$$A \in \mathbb{R}^{B\times T\times d}.$$

The MLP then expands each token representation to an intermediate width $m$ and projects back:

$$B \times T \times d \to B \times T \times m \to B \times T \times d.$$

| Stage | Logical shape | What changes? |
|---|---|---|
| Residual input | $B \times T \times d$ | nothing |
| Norm | $B \times T \times d$ | statistics/scale, not shape |
| Q | $B \times H_q \times T \times d_h$ | split into query heads |
| K,V | $B \times H_{kv} \times T \times d_h$ | KV heads may differ from Q |
| Attention output | $B \times T \times d$ | heads recombined/projected |
| MLP hidden | $B \times T \times m$ | feature expansion |
| Block output | $B \times T \times d$ | back on residual stream |

**Important.** The MLP width $m$ and attention head dimensions are temporary internal widths. Residual addition requires the branch output to return to $d$.

*Research anchors: [1], [11], [20]*

# 4 - 原 2017 年 LN 后区块

对应原文第 6 页；点击查看相邻原文对照页

4 - 原始 2017 POST-LN 块先残差，后 归一化（Normalization） 原始 Transformer 模型（Transformer） 在每个子层周围应用 残差连接（Residual Connection），然后标准化总和。瓦斯瓦尼等人。将子层输出写入 [1] 层归一化（LayerNorm）(x + Sublayer(x))。

对于 编码器（Encoder） 块，两个子层是 自注意力（Self-attention） 和位置 前馈网络（Feed-forward Network）。 解码器（Decoder） 插入 编码器（Encoder）-解码器（Decoder） 交叉注意力（Cross-attention） 的第三个子层。

对于两个子层的情况，简化的 Post-LN 块为 u = LN(x + Attn(x)), y = LN(u + FFN(u))。

这个顺序很重要。 归一化（Normalization）不仅仅是一个可以在不改变功能或优化行为的情况下移动的盒子。

原始 模型（Model） 还使用添加到 嵌入向量（Embeddings） 和 ReLU 前馈网络（Feed-forward Network）的正弦位置编码。这些选择在历史上很重要，但它们不是 Transformer 块（Transformer Block） 抽象定义的一部分。

理解框架（Mental Model）.Post-LN 表示残差和在成为下一个子层或块的输入之前被归一化。

研究锚点：[1]、[3] 理解 Transformer 块（Transformer Block）

# 4 - THE ORIGINAL 2017 POST-LN BLOCK

## Residual first, normalization after

The original Transformer applies a residual connection around each sublayer and then normalizes the sum. Vaswani et al. write the sublayer output as [1]

$$\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x)).$$

For an encoder block, the two sublayers are self-attention and the position-wise feed-forward network. The decoder inserts a third sublayer for encoder-decoder cross-attention.

For the two-sublayer case, a simplified Post-LN block is

$$u = \mathrm{LN}(x + \mathrm{Attn}(x)),$$

$$y = \mathrm{LN}(u + \mathrm{FFN}(u)).$$

This ordering matters. Normalization is not merely a box that can be moved without changing the function or optimization behavior.

The original model also used sinusoidal positional encodings added to embeddings and a ReLU feed-forward network. Those choices are historically important, but they are not part of the abstract definition of a Transformer block.

**Mental model.** Post-LN means the residual sum is normalized before it becomes the input to the next sublayer or block.

*Research anchors: [1], [3]*

# 5 - 现代 PRE-LN 解码器（Decoder） 块

对应原文第 7 页；点击查看相邻原文对照页

5 - 现代 PRE-LN 解码器（Decoder） 块标准化分支输入，保持 残差路径（Residual Path） 直接 GPT-2 将 层归一化（Layer Normalization） 移动到每个子块的输入，并在最后一个块之后添加最终的 归一化（Normalization）。 [4] GPT-3 保留了 GPT-2 之前的 归一化（Normalization） 架构。 [18] LLaMA 后来将 前置归一化（Pre-norm） 与 均方根归一化（RMSNorm）、SwiGLU 门控激活（SwiGLU） 和 RoPE 结合起来。 [8]

常见的串行 前置归一化（Pre-norm） 块为 u =x + Attn(Norm1(x)), y =u + MLP(Norm2(u))。

最终的 模型（Model） 级规范通常应用于语言 模型（Model） 输出头（Output Head） 之前的最后一个块之后，但这是 模型（Model） 架构选择，而不是每个单独块的一部分。

x 标准 注意力机制（Attention） + 标准 MLP +

重要。“现代 Transformer 模型（Transformer）”并不意味着一种通用的 前置归一化（Pre-norm） 配方。例如，Gemma 2 使用子层前和子层后 均方根归一化（RMSNorm）。 [16]

研究锚点：[4]、[8]、[16]、[18] 理解 Transformer 块（Transformer Block）

# 5 - THE MODERN PRE-LN DECODER BLOCK

### Normalize the branch input, keep the residual path direct

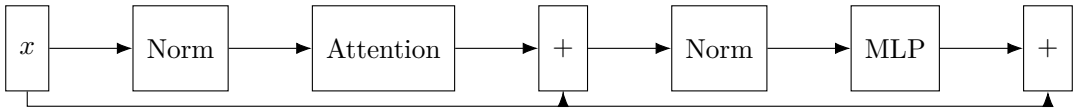
GPT-2 moved layer normalization to the input of each sub-block and added a final normalization after the last block. [4] GPT-3 retained GPT-2's pre-normalization architecture. [18] LLaMA later combined Pre-Norm with RMSNorm, SwiGLU, and RoPE. [8]

A common serial Pre-Norm block is

$$u = x + \mathrm{Attn}(\mathrm{Norm}_1(x)),$$

$$y = u + \mathrm{MLP}(\mathrm{Norm}_2(u)).$$

A final model-level norm is often applied after the last block before the language-model output head, but that is a model architecture choice rather than part of every individual block.




**Important.** "Modern Transformer" does not imply one universal Pre-Norm recipe. Gemma 2, for example, uses both pre- and post-sublayer RMSNorm. [16]

*Research anchors: [4], [8], [16], [18]*

# 6 - 为什么残留添加很重要

对应原文第 8 页；点击查看相邻原文对照页

6 - 为什么残差添加很重要 身份路径加上学习更新 考虑子层输出

导数包含一个恒等项：

这并不意味着梯度永远不会在深度 Transformer 模型（Transformer） 中消失或爆炸。 归一化（ Normalization），初始化、参数规模、优化器动态以及许多块的组成仍然很重要。但直接身份路径是残差架构可以保留和细化表示的结构原因，而不是强迫每一层从头开始重新学习它们。 [2]

残差连接（Residual Connections） 还施加形状规则。如果$x \in Rd$，则$F(x)$在加法点必须在$Rd$中。这就是为什么具有许多 头（Heads） 的 注意力机制（Attention） 分支以 输出投影（Output Projection） 结束，以及扩展至 widthm 的 MLP 以向下投影结束的原因。

重要： 残差连接（Residual Connection） 不是串联。它是相同形状张量的元素加法。串联会改变宽度并需要不同的下游参数。

研究锚：[2]

了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 6 - WHY THE RESIDUAL ADDITION MATTERS

## Identity path plus learned update

Consider a sublayer output

$$y = x + F(x).$$

The derivative contains an identity term:

$$\frac{\partial y}{\partial x} = I + \frac{\partial F(x)}{\partial x}.$$

This does not mean gradients can never vanish or explode in a deep Transformer. Normalization, initialization, parameter scale, optimizer dynamics, and the composition of many blocks still matter. But the direct identity path is the structural reason residual architectures can preserve and refine representations instead of forcing every layer to relearn them from scratch. [2]

Residual connections also impose shape discipline. If $x \in \mathbb{R}^d$, then $F(x)$ must be in $\mathbb{R}^d$ at the point of addition. That is why an attention branch with many heads ends in an output projection and why an MLP that expands to width $m$ ends with a down projection.

**Important.** A residual connection is not concatenation. It is elementwise addition of same-shaped tensors. Concatenation would change the width and require different downstream parameters.

*Research anchors: [2]*

# 7 - 层归一化（LayerNorm） 力学

对应原文第 9 页；点击查看相邻原文对照页

7 - 层归一化（LayerNorm） 力学 中心、缩放，然后应用学习的仿射参数 对于一个 词元（Token） 向量 x ∈ Rd，层归一化（LayerNorm） 计算标准化特征维度上的统计数据。在常见的一维情况下，

最初的 层归一化（LayerNorm） 论文在 归一化（Normalization） 之后介绍了每神经元的增益和偏差。[3] 当前的 PyTorch 层归一化（LayerNorm） 默认情况下同样支持可学习的元素缩放和偏差。 [19]

对于序列 模型（Models），层归一化（LayerNorm） 通常在其特征维度上独立应用于每个 词元（Token），而不是跨批次示例，也不跨序列位置。

理解框架（Mental Model）.层归一化（LayerNorm） 更改每个 词元（Token） 表示（Representation） 的坐标统计数据，但保持 张量形状（Tensor Shape） 不变。

研究锚点：[3]、[19] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 7 - LAYERNORM MECHANICS

## Center, scale, then apply learned affine parameters

For one token vector $x \in \mathbb{R}^d$, LayerNorm computes statistics across the normalized feature dimensions. In the common one-dimensional case,

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2,$$

$$\mathrm{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

The original LayerNorm paper introduced per-neuron gain and bias after normalization. [3] Current PyTorch `LayerNorm` likewise supports learnable elementwise scale and bias by default. [19]

For sequence models, LayerNorm is typically applied independently to each token over its feature dimension, not across batch examples and not across sequence positions.

**Mental model.** LayerNorm changes the coordinate statistics of each token representation but leaves the tensor shape unchanged.

*Research anchors: [3], [19]*

# 8 - 均方根归一化（RMSNorm） 力学

对应原文第 10 页；点击查看相邻原文对照页

8 - 均方根归一化（RMSNorm） 力学 按均方根缩放而不减去平均值 均方根归一化（RMSNorm） 删除了 层归一化（LayerNorm） 的显式重新居中步骤。 Forx∈Rd, RMS(x) =

均方根归一化（RMSNorm）(x)i =γi

均方根（x）。

张和 Sennrich 提出了 均方根归一化（RMSNorm），其假设是 层归一化（LayerNorm） 的重新居中不变性 在许多设置中是可有可无的。 [6] 当前 PyTorch 的 均方根归一化（RMSNorm） 遵循相同的 核心（Core） 操作并保留输入形状。 [20]

层归一化（LayerNorm） 均方根归一化（RMSNorm） 减去平均值 是 否 尺度统计 标准差 均方根 习得尺度 通常是 通常是 习得偏差 通常可用 不是 核心（Core） 的一部分 均方根归一化（RMSNorm） 公式 形状 变化 无 无 LLaMA 在 前置归一化（Pre-norm） Transformer 模型（Transformer） 中使用 均方根归一化（RMSNorm），Qwen3 同样如此报告 均方根归一化（RMSNorm） 和 pre-归一化（Normalization）。 [8,17]

研究锚点：[6]、[8]、[17]、[20] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 8 - RMSNORM MECHANICS

## Scale by root-mean-square without subtracting the mean

RMSNorm removes LayerNorm's explicit re-centering step. For $x \in \mathbb{R}^d$,

$$\mathrm{RMS}(x) = \sqrt{\epsilon + \frac{1}{d}\sum_{i=1}^{d} x_i^2},$$

$$\mathrm{RMSNorm}(x)_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}.$$

Zhang and Sennrich proposed RMSNorm on the hypothesis that LayerNorm's re-centering invariance is dispensable in many settings. [6] Current PyTorch's RMSNorm follows the same core operation and preserves input shape. [20]

| | LayerNorm | RMSNorm |
|---|---|---|
| Subtract mean | yes | no |
| Scale statistic | standard deviation | root mean square |
| Learned scale | usually yes | usually yes |
| Learned bias | commonly available | not part of core RMSNorm formula |
| Shape change | none | none |

LLaMA uses RMSNorm in a Pre-Norm Transformer, and Qwen3 likewise reports RMSNorm with pre-normalization. [8,17]

*Research anchors: [6], [8], [17], [20]*

# 9 - 工作正常示例

对应原文第 11 页；点击查看相邻原文对照页

9 - 工作规范示例 层归一化（LayerNorm） 和 均方根归一化（RMSNorm） 对相同的 向量（Vector） 执行不同的操作，忽略学习的增益/偏差并采取

对于 层归一化（LayerNorm），

为了可读性忽略 $\epsilon$，

归一化的 向量（Vector） 以零为中心。

对于 均方根归一化（RMSNorm），RMS(x) =

它的平均值不会被强制为零，因为该操作不会减去 μ。

重要： 层归一化（LayerNorm） 和 均方根归一化（RMSNorm） 不是具有不同名称的可互换公式。 模型（Model） 检查点使用特定的 归一化（Normalization） 规则和参数化进行训练。

研究锚点：[3]、[6] 了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 9 - WORKED NORM EXAMPLE

## LayerNorm and RMSNorm do different things to the same vector

Ignore learned gain/bias and take

$$x = [1, 2, 3, 4].$$

For LayerNorm,

$$\mu = 2.5, \qquad \sigma^2 = 1.25, \qquad \sigma \approx 1.1180.$$

Ignoring $\epsilon$ for readability,

$$\mathrm{LN}(x) \approx [-1.3416, -0.4472, 0.4472, 1.3416].$$

The normalized vector is centered around zero.

For RMSNorm,

$$\mathrm{RMS}(x) = \sqrt{\frac{1^2 + 2^2 + 3^2 + 4^2}{4}} = \sqrt{7.5} \approx 2.7386,$$

so

$$\mathrm{RMSNorm}(x) \approx [0.3651, 0.7303, 1.0954, 1.4606].$$

Its mean is not forced to zero because the operation never subtracts $\mu$.

**Important.** LayerNorm and RMSNorm are not interchangeable formulas with different names. A model checkpoint is trained with a specific normalization rule and parameterization.

*Research anchors: [3], [6]*

# 10 - LN 前 VS LN 后

对应原文第 12 页；点击查看相邻原文对照页

10 - PRE-LN VS POST-LN 相同的成分，不同的优化路径代数区别很紧凑：

LN 后：y = Norm(x +F (x))，LN 前：y =x +F (Norm(x))。

在 Post-LN 中，恒等路径在到达下一个块之前经过 归一化（Normalization）。在 Pre-LN 中，残差流（Residual Stream） 具有跨子层的直接附加路径。

熊等人。用平均场理论分析了这两个位置。在初始化时，他们发现原始 Post-LN Transformer 模型（Transformer） 中的输出层附近存在较大的预期梯度，从而激发了学习率预热，而 Pre-LN 公式在其分析和实验中具有更好的梯度表现。 [5]

该结果是关于优化行为的 证据（Evidence），而不是 Pre-LN 总是更好的普遍说法。后来的架构以不同的方式重新审视了 归一化（Normalization） 和残差 缩放（Scaling）。例如，DeepNorm 修改 残差连接（Residual Connections） 和初始化以稳定非常深的 Transformer。 [21]

理解框架（Mental Model）.归一化（Normalization） 放置会改变梯度遍历的计算图。

这不是外观上的重新排列。

研究锚点：[1]、[4]、[5]、[21] 理解 Transformer 块（Transformer Block）

# 10 - PRE-LN VS POST-LN

## Same ingredients, different optimization path

The algebraic distinction is compact:

$$\text{Post-LN:} \qquad y = \text{Norm}(x + F(x)),$$

$$\text{Pre-LN:} \qquad y = x + F(\text{Norm}(x)).$$

In Post-LN, the identity path passes through normalization before reaching the next block. In Pre-LN, the residual stream has a direct additive path across the sublayer.

Xiong et al. analyzed the two placements with mean-field theory. At initialization, they found large expected gradients near the output layers in the original Post-LN Transformer, motivating learning-rate warm-up, while the Pre-LN formulation had better-behaved gradients in their analysis and experiments. [5]

That result is evidence about optimization behavior, not a universal statement that Pre-LN is always better. Later architectures have revisited normalization and residual scaling in different ways. DeepNorm, for example, modifies residual connections and initialization to stabilize very deep Transformers. [21]

**Mental model.** Normalization placement changes the computational graph that gradients traverse. It is not a cosmetic rearrangement.

*Research anchors: [1], [4], [5], [21]*

# 11 - 注意力机制（Attention） 分支机构

对应原文第 13 页；点击查看相邻原文对照页

11 - 注意力机制（Attention） 分支 块的 词元（Token） 混合子层 对于隐藏状态 X∈ RB×T×d，自注意力（Self-attention） 首先形成 学习得到的投影（Learned Projections）。在传统的多头配方中，

重塑为 头（Heads） 后，缩放点积注意力（Scaled Dot-product Attention） 计算

其中 M 代表掩蔽或 注意力机制（Attention） 偏差。然后头输出被重新组合并通过 WO 投影回剩余添加之前的 widthd。 [1]

注意力机制（Attention）分支是词元（Token）仓位可以交换信息的地方。这使得它与 MLP 分支有根本的不同，MLP 分支独立地将相同的特征变换应用于每个 词元（Token） 位置。

重要：残差分支接收注意力输出投影，而不是注意力概率 矩阵（Matrix）。 注意力权重（Attention Weights） 是中间体。

研究锚点：[1]、[22] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 11 - THE ATTENTION BRANCH

## The block's token-mixing sublayer

For hidden states $X \in \mathbb{R}^{B\times T\times d}$, self-attention first forms learned projections. In a conventional multi-head formulation,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

After reshaping into heads, scaled dot-product attention computes

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V,$$

where $M$ represents masking or attention bias. Head outputs are then recombined and projected through $W_O$ back to width $d$ before the residual addition. [1]

The attention branch is where token positions can exchange information. That makes it fundamentally different from the MLP branch, which applies the same feature transformation independently to each token position.

**Important.** The residual branch receives the attention *output projection*, not the attention-probability matrix. Attention weights are an intermediate.

*Research anchors: [1], [22]*

# 12 - 绳索进入的位置

对应原文第 14 页；点击查看相邻原文对照页

12 - WHERE ROPE ENTERS 位置修改 注意力机制（Attention） 内部的 Q/K 兼容性 现代仅限 解码器（Decoder） 的法学硕士通常使用 旋转位置嵌入（Rotary Position Embedding） (RoPE)。 RoPE 不会通过添加位置 向量（Vector） toX 来更改 残差流（Residual Stream）。相反，标准 RoPE 在评估 查询（Query） 和 键（Key） 组件的点积之前对其应用位置相关的旋转。

因此，在 前置归一化（Pre-norm） 块中，概念顺序为 X→ Norm(X)→Q,K,V → RoPE(Q,K )→ 注意力机制（Attention）。

LLaMA 在每一层明确用 RoPE 替换绝对位置 嵌入向量（Embeddings）。 [8]

这种区别很重要，因为它将三个经常折叠在一起的对象分开：

- 残差表示X；

- 从X 导出的学习的Q/K/V 投影；

- 应用于Q 和K 的位置相关变换。

重要。 RoPE 并未“添加到 LLaMA 样式块中的 词元嵌入（Token Embedding）”。它改变了 注意力机制（Attention） 内部的 Q/K 交互。

研究锚点：[8]、[12] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 12 - WHERE ROPE ENTERS

## Position modifies Q/K compatibility inside attention

Modern decoder-only LLMs often use Rotary Position Embedding (RoPE). RoPE does not alter the residual stream by adding a positional vector to $X$. Instead, standard RoPE applies position-dependent rotations to query and key components before their dot products are evaluated. [12]

In a Pre-Norm block, the conceptual order is therefore

$$X \to \mathrm{Norm}(X) \to Q, K, V \to \mathrm{RoPE}(Q, K) \to \mathrm{Attention}.$$

LLaMA explicitly replaced absolute positional embeddings with RoPE at each layer. [8]

This distinction matters because it separates three objects that are often collapsed together:

- the residual representation $X$;
- the learned Q/K/V projections derived from $X$;
- the position-dependent transformation applied to Q and K.

**Important.** RoPE is not "added to the token embedding" in LLaMA-style blocks. It changes the Q/K interaction inside attention.

*Research anchors: [8], [12]*

# 13 - 块内的 MHA、GQA、MQA 和 MLA

对应原文第 15 页；点击查看相邻原文对照页

13 - 块内的 MHA、GQA、MQA 和 MLA 当 注意力机制（Attention） 表示（Representation） 更改时，块骨架保持不变 残差拓扑不会告诉您存在多少个 键值头（Key/Value Heads） 或在解码期间缓存了哪些 表示（Representation）。

与 Hq 查询头（Query Heads） 和 HKv 键值头（Key/Value Heads） 一起，

MHA 通常使用Hkv =Hq。 MQA使用1个KV头。 GQA 使用中间数。 [11]

例如，Qwen3 在其密集的 模型（Models） 中报告 GQA。 [17]

MLA 采用不同的路线：DeepSeek-V2 将 键（Key）/值（Value） 信息压缩为低维 潜在表示（Latent Representation） 并单独处理 RoPE 敏感组件。 [14] 然而，在 注意力机制（Attention） 计算之后，该分支仍然返回 残差流（Residual Stream） 的宽度 d 更新。

理解框架（Mental Model）。 MHA/GQA/MQA/MLA 更改了 注意力机制（Attention） 分支的内部结构。它们不会取代块周围的范数残差 MLP 结构。

研究锚点：[11]、[14]、[17] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 13 - MHA, GQA, MQA, AND MLA INSIDE THE BLOCK

## The block skeleton stays while the attention representation changes

The residual topology does not tell you how many key/value heads exist or what representation is cached during decoding.

With $H_q$ query heads and $H_{kv}$ key/value heads,

$$Q \in \mathbb{R}^{B\times H_q\times T\times d_h}, \quad K, V \in \mathbb{R}^{B\times H_{kv}\times T\times d_h}.$$

MHA typically uses $H_{kv} = H_q$. MQA uses one KV head. GQA uses an intermediate number. [11] Qwen3, for example, reports GQA across its dense models. [17]

MLA takes a different route: DeepSeek-V2 compresses key/value information into a lower-dimensional latent representation and handles RoPE-sensitive components separately. [14] Yet after its attention computation, the branch still returns a width-$d$ update to the residual stream.

**Mental model.** MHA/GQA/MQA/MLA change the internals of the attention branch. They do not replace the surrounding norm-residual-MLP structure of the block.

*Research anchors: [11], [14], [17]*

# 14 - 输出投影（Output Projection） 关闭 注意力机制（Attention） 分支机构

对应原文第 16 页；点击查看相邻原文对照页

14 - 输出投影（Output Projection） 关闭 注意力机制（Attention） 分支 头（Heads） 必须返回到剩余宽度 假设每个 Hq 查询头（Query Heads） 生成一个维度为 dh 的 向量（Vector）。连接头输出会产生宽度为 Hqdh 的 表示（Representation）。在常见情况下Hqdh = d，连接的张量（Tensor）已经具有宽度d，但它仍然通过学习的输出投影（Output Projection）传递

然后分支返回

明确的。 [1]

输出投影（Output Projection） 不仅仅是一次重塑。这是通过学习将头输出混合回 模型（Model） 的剩余特征空间。

重要：只计算WQ、W K、W V就漏掉了标准MHA:WO中的四大投影矩阵（Projection Matrices）之一。

研究锚：[1]

了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 14 - THE OUTPUT PROJECTION CLOSES THE ATTENTION BRANCH

## Heads must return to the residual width

Suppose each of $H_q$ query heads produces a vector of dimension $d_h$. Concatenating head outputs yields a representation of width $H_q d_h$. In the common case $H_q d_h = d$, the concatenated tensor already has width $d$, but it is still passed through a learned output projection

$$W_O \in \mathbb{R}^{d \times d}.$$

The branch then returns

$$A = \text{Concat}(\text{head}_1, \ldots, \text{head}_{H_q}) W_O$$

with shape $B \times T \times d$, making

$$x + A$$

well-defined. [1]

The output projection is not merely a reshape. It is a learned mixing of head outputs back into the model's residual feature space.

**Important.** Counting only $W_Q, W_K, W_V$ misses one of the four major projection matrices in standard MHA: $W_O$.

*Research anchors: [1]*

# 15 - MLP 是位置明智的

对应原文第 17 页；点击查看相邻原文对照页

15 - MLP 是位置明智的 没有 词元（Token） 混合的特征混合 原始的 Transformer 模型（Transformer） 将其第二个子层称为位置明智的 前馈网络（Feed-forward Network），因为相同的非线性函数独立应用于每个序列位置。 [1]

然后

词元（Token） 职位。所有可用的跨 词元（Token） 信息必须已经存在于 词元（Token） 的当前 隐藏状态（Hidden State） 中，通常是在 注意力机制（Attention） 跨序列混合信息之后。

这为区块提供了有用的分工：

- 注意力机制（Attention） 混合不同头寸的信息；

- MLP 在每个位置的特征 向量（Vector） 内转换信息。

理解框架（Mental Model）。“位置明智”意味着共享功能，跨词元（Token）位置独立应用。这并不意味着 MLP 不知道上下文：它的输入状态已经可以包含早期 注意力机制（Attention） 混合的上下文。

研究锚：[1]

了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 15 - THE MLP IS POSITION-WISE

## Feature mixing without token mixing

The original Transformer calls its second sublayer a position-wise feed-forward network because the same nonlinear function is applied independently to each sequence position. [1]

If $X \in \mathbb{R}^{B\times T\times d}$ and

$$f : \mathbb{R}^d \to \mathbb{R}^d,$$

then

$$Y_{b,t,:} = f(X_{b,t,:})$$

for every batch element $b$ and token position $t$. The MLP does not directly read neighboring token positions. All cross-token information available to it must already be present in that token's current hidden state, typically after attention has mixed information across the sequence.

This gives the block a useful division of labor:

- attention mixes information *across positions*;
- the MLP transforms information *within each position's feature vector*.

**Mental model.** "Position-wise" means shared function, independent application across token positions. It does not mean the MLP is unaware of context: its input state can already contain context mixed in by earlier attention.

*Research anchors: [1]*

# 16 - 原始 RELU 前馈网络（Feed-forward Network）

对应原文第 18 页；点击查看相邻原文对照页

16 - 原始 RELU 前馈网络（Feed-forward Network） 扩展、激活、投影回 2017 Transformer 模型（Transformer） 使用两个线性变换，它们之间有一个 ReLU：[1]

FFN(x) = max(0,xW 1 +b1)W2 +b2。

具有剩余宽度d和前馈（Feed-forward）宽度m，

原始基数 Transformer 模型（Transformer） 使用 d = 512 和 m = 2048，进行 4 倍扩展。

忽略偏差，这个密集 FFN 的参数计数为

形式 = 4d,

这就是为什么前馈分支可以包含大部分 adenseTransformer 参数，即使它从不混合 词元（Token） 位置。

重要： 4d 比例是原始架构的设计选择，而不是变形金刚的数学要求。现代门控 MLP 通常使用不同的扩展比。

研究锚：[1]

了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 16 - THE ORIGINAL RELU FEED-FORWARD NETWORK

## Expand, activate, project back

The 2017 Transformer uses two linear transformations with a ReLU between them: [1]

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

With residual width $d$ and feed-forward width $m$,

$$W_1 \in \mathbb{R}^{d\times m}, \qquad W_2 \in \mathbb{R}^{m\times d}.$$

The original base Transformer used $d = 512$ and $m = 2048$, a $4\times$ expansion.

Ignoring biases, the parameter count of this dense FFN is

$$P_{\mathrm{FFN}} = 2dm.$$

For $m = 4d$,

$$P_{\mathrm{FFN}} = 8d^2.$$

That is why the feed-forward branch can contain a large fraction of a dense Transformer's parameters even though it never mixes token positions.

**Important.** The $4d$ ratio is a design choice from the original architecture, not a mathematical requirement of Transformers. Modern gated MLPs often use different expansion ratios.

*Research anchors: [1]*

# 17 - 从 GELU 到门控 MLPS

对应原文第 19 页：点击查看相邻原文对照页

17 - 从 GELU 到门控 MLPS 非线性是架构的一部分 Transformer 模型（Transformer） MLP 在 ReLU 中并未保持固定。后来的 Transformer 模型（Transformer） 语言 模型（Models） 采用了 GELU 系列 激活（Activations），而后续架构也采用了门控线性单元变体。

通用门控单元使用两个 学习得到的投影（Learned Projections） 并将它们按元素组合：

门(x) = phi(xWg)⊙(xWu)。

第三个投影将门控 隐藏表示（Hidden Representation） 映射回 widthd。

Shazeer 系统地比较了 Transformer 模型（Transformer） 前馈（Feed-forward） 子层中的 GLU 变体，发现了几种门控变体，包括 SwiGLU 门控激活（SwiGLU） 和 GEGLU，与研究设置中常用的 ReLU 或 GELU 基线相比，质量得到了提高。 [7]

激活（Activation） 选择更改了多个函数调用。 门控（Gating） 引入了额外的 投影矩阵（Projection Matrix），因此更改了参数计算、计算以及用于计算等效比较的隐藏宽度选择。

研究锚点：[7]、[23] 理解 Transformer 块（Transformer Block）

# 17 - FROM GELU TO GATED MLPS

## The nonlinearity is part of the architecture

Transformer MLPs did not remain fixed at ReLU. Later Transformer language models adopted GELU-family activations, while subsequent architectures also adopted gated linear-unit variants. A generic gated unit uses two learned projections and combines them elementwise:

$$\mathrm{Gate}(x) = \phi(xW_g) \odot (xW_u).$$

A third projection maps the gated hidden representation back to width $d$.

Shazeer systematically compared GLU variants in Transformer feed-forward sublayers and found several gated variants, including SwiGLU and GEGLU, improved quality over the commonly used ReLU or GELU baselines in the studied settings. [7]

The activation choice changes more than one function call. Gating introduces an additional projection matrix and therefore changes parameter accounting, compute, and the hidden-width choice used for compute-equivalent comparisons.

*Research anchors: [7], [23]*

# 18 - SwiGLU 门控激活（SwiGLU） 力学

对应原文第 20 页；点击查看相邻原文对照页

18 - SwiGLU 门控激活（SwiGLU） 力学 两个向上投影，一个门控乘积，一个向下投影 常见的 SwiGLU 门控激活（SwiGLU） MLP 可以写为 g = SiLU(xWg), u =xWu, h =g⊙u, y =hWd，其中 Wg,W u∈ Rd×m 且 Wd∈ Rm×d。 SiLU（也称为 Swish，β = 1）为 SiLU(z) = zσ (z)。

元素乘积通过另一个变换流来门控一个投影流。

LLaMA 明确用 SwiGLU 门控激活（SwiGLU） 替换 ReLU，并相对于 4d FFN 减少隐藏宽度以解释第三个矩阵（Matrix）。 [8] PaLM 同样使用 SwiGLU 门控激活（SwiGLU） 并强调计算等效比较。 [10]

丝路

研究锚点：[7]、[8]、[10] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 18 - SWIGLU MECHANICS

## Two upward projections, one gated product, one downward projection

A common SwiGLU MLP can be written

$$g = \mathrm{SiLU}(xW_g), \qquad u = xW_u,$$

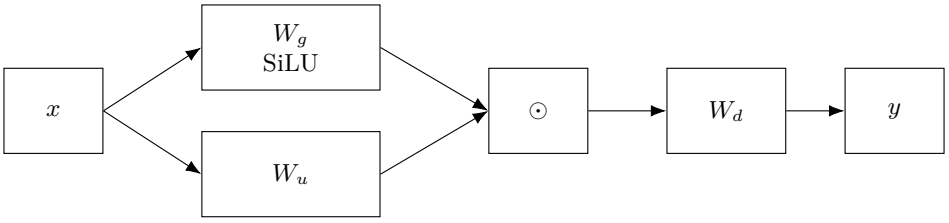
$$h = g \odot u, \qquad y = hW_d,$$

where $W_g, W_u \in \mathbb{R}^{d\times m}$ and $W_d \in \mathbb{R}^{m\times d}$. SiLU (also called Swish with $\beta = 1$) is

$$\mathrm{SiLU}(z) = z\,\sigma(z).$$

The elementwise product gates one projected stream by another transformed stream.

LLaMA explicitly replaced ReLU with SwiGLU and reduced the hidden width relative to a $4d$ FFN to account for the third matrix. [8] PaLM likewise used SwiGLU and emphasized compute-equivalent comparisons. [10]




*Research anchors: [7], [8], [10]*

# 19 - 为什么门控 MLPS 使用不同的宽度

对应原文第 21 页；点击查看相邻原文对照页

19 - 为什么门控 MLPS 使用不同的宽度 在比较 激活（Activations） 之前匹配参数预算 两个 矩阵（Matrix） FFN 大约有

三个 矩阵（Matrix） SwiGLU 门控激活（SwiGLU） 式 FFN 大约有

如果 ReLU 基线（Baseline） 使用 mReLU = 4d，则匹配参数计数给出 2d(4d) = 3dm SwiGLU 门控激活（SwiGLU），

这就是将 a4d ReLU FFN 与 a4d SwiGLU 门控激活（SwiGLU） FFN 进行比较并将每个差异归因于 激活（Activation） 具有误导性的原因：后者具有超大的 投影矩阵（Projection Matrix）。

LLaMA 在特定于架构的 舍入（Rounding） 之前将其 SwiGLU 门控激活（SwiGLU） 维度描述为 2 3 (4d)。[8]

理解框架（Mental Model）。当 投影矩阵（Projection Matrices） 的数量发生变化时，隐藏宽度必须是任何公平参数/计算比较的一部分。

研究锚点：[7]、[8]、[10] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 19 - WHY GATED MLPS USE A DIFFERENT WIDTH

## Match parameter budget before comparing activations

A two-matrix FFN has approximately

$$P_{\text{FFN}} = 2dm.$$

A three-matrix SwiGLU-style FFN has approximately

$$P_{\text{SwiGLU}} = 3dm.$$

If the ReLU baseline uses $m_{\text{ReLU}} = 4d$, then matching parameter count gives

$$2d(4d) = 3d\, m_{\text{SwiGLU}},$$

so

$$m_{\text{SwiGLU}} = \frac{8}{3}d \approx 2.67d.$$

This is the reason it is misleading to compare a $4d$ ReLU FFN against a $4d$ SwiGLU FFN and attribute every difference to the activation: the latter has an extra large projection matrix.

LLaMA describes its SwiGLU dimension as $\frac{2}{3}(4d)$ before architecture-specific rounding. [8]

**Mental model.** When the number of projection matrices changes, the hidden width must be part of any fair parameter/compute comparison.

*Research anchors: [7], [8], [10]*

# 20 - 工作参数示例

对应原文第 22 页；点击查看相邻原文对照页

20 - 工作参数示例 为什么 MLP 通常主导密集块的参数 取一个说明性块，其中 d = 768、H q = Hkv = 12、d h = 64。

对于标准 MHA，忽略偏差，Pattn = 4d2 = 4(768) 2 = 2,359,296，涵盖 WQ、W K、W V、W O。

m = 4d = 3072 的 ReLU FFN 有

所以这两个主要分支包含大约

规范和偏差之前的投影参数，其中 FFN 占总数的三分之二。

现在选择一个参数匹配的SwiGLU 门控激活（SwiGLU） widthm = 2048 = 8

门控 MLP 具有与本示例中的 4d 个两个 矩阵（Matrix） FFN 完全相同的 矩阵（Matrix） 参数计数。

重要：这是一个说明性的密集 MHA 块。 GQA/MQA 减少了 K/V 投影参数，MoE 更改了总参数与活动参数计算。

研究锚点：[1]、[7]、[8]、[11] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 20 - WORKED PARAMETER EXAMPLE

## Why the MLP often dominates a dense block's parameters

Take an illustrative block with

$$d = 768, \qquad H_q = H_{kv} = 12, \qquad d_h = 64.$$

For standard MHA, ignoring biases,

$$P_{\text{attn}} = 4d^2 = 4(768)^2 = 2{,}359{,}296,$$

covering $W_Q, W_K, W_V, W_O$.

A ReLU FFN with $m = 4d = 3072$ has

$$P_{\text{FFN}} = 2(768)(3072) = 4{,}718{,}592.$$

So the two major branches contain about

$$7{,}077{,}888$$

projection parameters before norms and biases, with the FFN accounting for two-thirds of that total.

Now choose a parameter-matched SwiGLU width $m = 2048 = \frac{8}{3}d$:

$$P_{\text{SwiGLU}} = 3(768)(2048) = 4{,}718{,}592.$$

The gated MLP has exactly the same matrix-parameter count as the $4d$ two-matrix FFN in this example.

**Important.** This is an illustrative dense MHA block. GQA/MQA reduce K/V projection parameters, and MoE changes total vs active parameter accounting.

*Research anchors: [1], [7], [8], [11]*

# 21 - 加工形状追踪

对应原文第 23 页；点击查看相邻原文对照页

21 - 加工形状追踪 一个块，端到端

剩余输入为

归一化（Normalization） 保留 [2, 4, 8]。 Q/K/V 重塑为

对于 自注意力（Self-attention），每人得分 张量（Tensor） 具有形状

因为 4 个 查询（Query） 位置中的每一个都得分 4 个 键（Key） 位置。经过 值（Value） 聚合和头连接后，注意力机制（Attention） 分支返回

因此，第一个剩余添加仍然是[2,4,8]。

MLP 独立扩展每个 词元（Token）：

第二个残差添加再次将块输出留在

理解框架（Mental Model）。通过普通块，序列长度保持T。 注意力机制（Attention） 改变位置交互的方式，而剩余宽度在每个分支后返回 tod。

研究锚：[1]

了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 21 - WORKED SHAPE TRACE

## One block, end to end

Take

$$B = 2, \quad T = 4, \quad d = 8, \quad H_q = H_{kv} = 2, \quad d_h = 4, \quad m = 32.$$

The residual input is

$$X : [2, 4, 8].$$

Normalization keeps $[2, 4, 8]$. Q/K/V reshape to

$$[2, 2, 4, 4].$$

For self-attention, the per-head score tensor has shape

$$[2, 2, 4, 4],$$

because each of 4 query positions scores 4 key positions. After value aggregation and head concatenation, the attention branch returns

$$[2, 4, 8].$$

The first residual add therefore remains $[2, 4, 8]$.

The MLP expands each token independently:

$$[2, 4, 8] \to [2, 4, 32] \to [2, 4, 8].$$

The second residual add again leaves the block output at

$$[2, 4, 8].$$

**Mental model.** The sequence length stays $T$ through an ordinary block. Attention changes how positions interact, while residual width returns to $d$ after every branch.

*Research anchors: [1]*

# 22 - 串行与并行剩余块

对应原文第 24 页；点击查看相邻原文对照页

22 - 串行与并行残余块 注意力机制（Attention） 和 MLP 不需要严格顺序 熟悉的 前置归一化（Pre-norm） 块是串行的：MLP 在 注意力机制（Attention） 更新后接收残余状态。以简化形式，u =x + Attn(Norm(x)) ，y =u + MLP(Norm(u))。

PaLM 使用并行公式，其中 注意力机制（Attention） 和 MLP 读取相同的归一化残差输入，并将它们的输出加在一起：[10]

y =x + Attn(Norm(x)) + MLP(Norm(x))。

PaLM 论文报告称，这使得 注意力机制（Attention） 和 MLP 输入 矩阵（Matrix） 乘法的融合成为可能，并在其设置中大规模地产生了约 15% 的快 训练（Training），同时它们的消融显示出与规模相关的质量效应。 [10]

重要。并行残差块在代数上与串行块不同。 MLP 在生成自己的更新之前不再看到当前块的 注意力机制（Attention） 更新。

研究主播：[10]

了解 Transformer 块（Transformer Block）

# 22 - SERIAL VS PARALLEL RESIDUAL BLOCKS

## Attention and MLP need not be strictly sequential

The familiar Pre-Norm block is serial: the MLP receives the residual state after the attention update. In simplified form,

$$u = x + \mathrm{Attn}(\mathrm{Norm}(x)),$$

$$y = u + \mathrm{MLP}(\mathrm{Norm}(u)).$$

PaLM used a parallel formulation in which attention and the MLP read the same normalized residual input and their outputs are added together: [10]

$$y = x + \mathrm{Attn}(\mathrm{Norm}(x)) + \mathrm{MLP}(\mathrm{Norm}(x)).$$

The PaLM paper reports that this enabled fusion of the attention and MLP input matrix multiplications and produced roughly 15% faster training at large scale in their setup, while their ablations showed scale-dependent quality effects. [10]

**Important.** Parallel residual blocks are not algebraically identical to serial blocks. The MLP no longer sees the current block's attention update before producing its own update.

*Research anchors: [10]*

# 23 - 编码器（Encoder） 和 解码器（Decoder） 块系列

对应原文第 25 页；点击查看相邻原文对照页

23 - 编码器（Encoder） 和解码器（Decoder） 块系列 子层取决于模型（Model） 系列。2017 Transformer 模型（Transformer） 使用两个不同的堆栈。 [1]

模块系列 自注意力（Self-attention） 交叉注意力（Cross-attention） MLP 编码器（Encoder） 双向/全 否 是 解码器（Decoder）-only LM 因果 否 是 编码器（Encoder）-解码器（Decoder） 解码器（Decoder） 因果 是，超过 编码器（Encoder） 输出 是交叉注意力（Cross-attention） 添加了另一个剩余子层，因为它的 查询（Queries） 来自 解码器（Decoder） 状态，而它的 键（Keys）/值（Values） 来自 编码器（Encoder） 表示。 仅解码器大语言模型（Decoder-only LLM） 完全省略了 交叉注意力（Cross-attention） 子层。

现代多模式系统可以重新引入 交叉注意力（Cross-attention） 或其他特定于模式的块，但“Transformer 块（Transformer Block）”应始终相对于正在讨论的架构进行解释。

理解框架（Mental Model）。 自注意力（Self-attention） 描述了 Q/K/V 的来源。因果关系描述了哪些位置可以相互作用。 交叉注意力（Cross-attention） 描述了 K/V 的单独来源。

研究锚：[1]

了解 Transformer 块（Transformer Block）

# 23 - ENCODER AND DECODER BLOCK FAMILIES

## The sublayers depend on the model family

The 2017 Transformer uses two different stacks. [1]

| Block family | Self-attention | Cross-attention | MLP |
|---|---|---|---|
| Encoder | bidirectional/full | no | yes |
| Decoder-only LM | causal | no | yes |
| Encoder-decoder decoder | causal | yes, over encoder output | yes |

Cross-attention adds another residual sublayer because its queries come from the decoder state while its keys/values come from encoder representations. A decoder-only LLM omits that cross-attention sublayer entirely.

Modern multimodal systems can reintroduce cross-attention or other modality-specific blocks, but "Transformer block" should always be interpreted relative to the architecture being discussed.

**Mental model.** Self-attention describes where Q/K/V come from. Causality describes which positions can interact. Cross-attention describes a separate source for K/V.

*Research anchors: [1]*

# 24 - 因果屏蔽位于 注意力机制（Attention） 内部

对应原文第 26 页；点击查看相邻原文对照页

24 - 注意力机制（Attention） 内部存在因果屏蔽 块拓扑和连接性是不同的问题 仅 解码器（Decoder） 块通常称为“因果 Transformer 块（Transformer Block）”，但因果关系是在 注意力机制（Attention） 计算内部强制执行的。对于 查询（Query） 位置 i 和 键（Key） 位置 j，加性掩码（Additive Mask） 可以使用 Mij =

然后 注意力机制（Attention） 计算

周围的残差/MLP 拓扑本身不会成为因果关系。 MLP 是位置式的，因此不会产生跨位置泄漏，但 注意力机制（Attention） 子层必须强制执行允许的连接。

当前 PyTorch 缩放的 点积注意力（Dot-product Attention） 将因果行为公开为 注意力机制（Attention） 操作，并记录非方形情况下单独的 查询（Query） 和源长度。 [22]

重要。“解码器（Decoder） 块”并不是“到处都是下三角 矩阵（Matrix）”的同义词。因果限制属于 查询（Query） 和 键（Key） 位置之间的 注意力机制（Attention） 关系。

研究锚点：[1]、[22] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 24 - CAUSAL MASKING IS INSIDE ATTENTION

## Block topology and connectivity are different questions

A decoder-only block is often called a "causal Transformer block," but causality is enforced inside the attention computation. For query position $i$ and key position $j$, an additive mask can use

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Then attention computes

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right) V.$$

The surrounding residual/MLP topology does not become causal by itself. The MLP is position-wise, so it does not create cross-position leakage, but the attention sublayer must enforce the permitted connectivity.

Current PyTorch scaled-dot-product attention exposes causal behavior as an attention operation and documents separate query and source lengths for non-square cases. [22]

**Important.** "Decoder block" is not synonymous with "lower-triangular matrix everywhere." The causal restriction belongs to the attention relation between query and key positions.

*Research anchors: [1], [22]*

# 25 - MOE 取代 MLP 分支

对应原文第 27 页；点击查看相邻原文对照页

25 - MOE 替换 MLP 分支 稀疏 专家（Experts） 更改条件计算，而不是块骨架 在许多稀疏 混合专家（Mixture-of-Experts） Transformer 中，密集的 MLP 子层被路由的 专家（Expert） 库替换。对于 词元（Token） statex，路由器（Router） 选择一小组 专家（Experts） 并组合它们的输出：

MoE 分支仍然必须返回 widthd 的 向量（Vector），以便剩余更新保持有效：

x←x + MoE(范数(x))。

这就是为什么 MoE 可以理解为对 前馈（Feed-forward） 分支的更改，而不是完全不同的 Transformer 模型（Transformer） 拓扑。

例如，Qwen3 的 MoE 模型（Models） 围绕 注意力机制（Attention）/norms 共享相同的基本密集 模型（Model） 架构，但用细粒度路由的 专家（Experts） 替换 前馈（Feed-forward） 计算。 [17]

重要。 MoE 稀疏度和 注意力机制（Attention） 稀疏度不同。 MoE 稀疏哪些 前馈（Feed-forward） 参数对于 词元（Token） 有效； 稀疏注意力（Sparse Attention） 更改 词元（Token） 对相互作用。

研究锚点：[17]、[24] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 25 - MOE REPLACES THE MLP BRANCH

### Sparse experts change conditional computation, not the block skeleton

In many sparse Mixture-of-Experts Transformers, the dense MLP sublayer is replaced by a routed expert bank. For token state $x$, a router selects a small set of experts and combines their outputs:

$$y_{\text{MoE}} = \sum_{i \in \text{Top-}k} \tilde{p}_i(x) E_i(x).$$

The MoE branch still must return a vector of width $d$ so the residual update remains valid:

$$x \leftarrow x + \text{MoE}(\text{Norm}(x)).$$

This is why MoE can be understood as a change to the feed-forward branch rather than a wholly different Transformer topology.

Qwen3's MoE models, for example, share the same fundamental dense-model architecture around attention/norms but replace the feed-forward computation with fine-grained routed experts. [17]

**Important.** MoE sparsity and attention sparsity are different. MoE sparsifies which feed-forward parameters are active for a token; sparse attention changes which token pairs interact.

*Research anchors: [17], [24]*

# 26 - QK-NORM 不是块 归一化（Normalization）

对应原文第 28 页；点击查看相邻原文对照页

26 - QK-NORM 不是块 归一化（Normalization） 归一化（Normalization） 也可以出现在 注意力机制（Attention） 分支内 前置归一化（Pre-norm） 或 Post-LN 描述了范数相对于剩余分支的位置。现代 模型（Models） 还可以标准化内部 注意力机制（Attention） 张量。

Qwen3 在块级别报告 均方根归一化（RMSNorm） 和 pre-归一化（Normalization），并分别在 注意力机制（Attention） 机制中报告 QK-Norm，以稳定 训练（Training）。[17] 这些是不同的 归一化（Normalization） 站点：

- 块归一化：在子层之前或之后变换残差流输入；

- QK-Norm：在分数计算之前对 注意力机制（Attention） 内的 查询（Query）/键（Key） 表示进行标准化。

移动或移除其中一个不能被视为是另一个。

理解框架（Mental Model）.A 模型（Model） 可以包含具有不同张量、轴、用途和参数的多个归一化。“使用 均方根归一化（RMSNorm）”不足以重建块。

研究锚：[17]

了解 Transformer 块（Transformer Block）

# 26 - QK-NORM IS NOT BLOCK NORMALIZATION

### Normalization can appear inside the attention branch too

Pre-Norm or Post-LN describes where a norm sits relative to the residual branch. Modern models can also normalize internal attention tensors.

Qwen3 reports RMSNorm with pre-normalization at the block level and, separately, QK-Norm in the attention mechanism to stabilize training. [17] These are different normalization sites:

- **block norm**: transforms the residual-stream input before or after a sublayer;
- **QK-Norm**: normalizes query/key representations inside attention before score computation.

Moving or removing one cannot be reasoned about as if it were the other.

**Mental model.** A model can contain several normalizations with different tensors, axes, purposes, and parameters. “Uses RMSNorm” is not enough to reconstruct the block.

*Research anchors: [17]*

# 27 - GEMMA 2：一个有用的反例

对应原文第 29 页；点击查看相邻原文对照页

27 - GEMMA 2：一个有用的反例 现代模块并不都集中到一个规范位置 人们很容易将当代 LLM 模块总结为“Pre-均方根归一化（RMSNorm） + 注意力机制（Attention） + SwiGLU 门控激活（SwiGLU）”。Gemma 2 是不普遍化该模式的一个有用的理由。

其技术报告指定了前置归一化（Pre-norm）和后置归一化（Post-norm）以及均方根归一化（RMSNorm）：每个注意力机制（Attention）和前馈（Feed-forward）子层的输入和输出都被标准化。它还使用 GeGLU 而不是 SwiGLU 门控激活（SwiGLU）、GQA 和交替的局部/全局注意力（Global Attention） 层。 [16]

这意味着 Gemma 2 样式子层在概念上更接近 y =x + PostNorm(F (PreNorm(x)))，具体实现细节由 模型（Model） 确定。

重要：“Transformer 模型（Transformer）”一词标识了一个系列，而不是冻结的 2017 年接线图。

准确的解释应该说明正在描述哪种架构。

研究锚：[16]

了解 Transformer 块（Transformer Block）

# 27 - GEMMA 2: A USEFUL COUNTEREXAMPLE

## Modern blocks do not all converge to one norm placement

It is tempting to summarize contemporary LLM blocks as "Pre-RMSNorm + attention + SwiGLU." Gemma 2 is a useful reason not to universalize that pattern.

Its technical report specifies both pre-norm and post-norm with RMSNorm: the input and output of each attention and feed-forward sublayer are normalized. It also uses GeGLU rather than SwiGLU, GQA, and alternating local/global attention layers. [16]

That means a Gemma 2-style sublayer is closer conceptually to

$$y = x + \mathrm{PostNorm}(F(\mathrm{PreNorm}(x))),$$

with exact implementation details determined by the model.

**Important.** The word "Transformer" identifies a family, not a frozen 2017 wiring diagram. Accurate explanations should state which architecture is being described.

*Research anchors: [16]*

# 28 - 训练（Training）、预填充（Prefill） 和解码

对应原文第 30 页；点击查看相邻原文对照页

28 - 训练（Training）、预填充（Prefill） 和解码 相同的块参数，截然不同的执行形状 模型（Model） 在开始生成 词元（Tokens） 时不会切换到不同的数学块。

改变的是作品的形态以及什么状态可以重用。

训练（Training）/完整序列 提示 预填充（Prefill） 增量解码 查询（Query） 位置很多很多，通常是一个/几个新位置 过去在通道中计算的 K/V 构建缓存 从缓存读取 MLP 位置 多个并行 多个并行 新位置 LM 的序列依赖性 因果掩码（Causal Mask） 因果掩码（Causal Mask） 历史记录已缓存 在解码期间，最新的位置词元（Token） 仍然经过每个 Transformer 块（Transformer Block）。在每个 注意力机制（Attention） 子层，其 查询（Query） 负责缓存的 K/V 历史记录。 MLP 保持位置方式，只需要处理该解码步骤的新词元（Token） 状态。

理解框架（Mental Model）。 KV 缓存改变了 注意力机制（Attention） 分支重用历史记录的方式。它不会跳过新生成的 词元（Token） 块的其余部分。

研究锚点：[4]、[22] 理解 Transformer 块（Transformer Block）

# 28 - TRAINING, PREFILL, AND DECODE

### Same block parameters, very different execution shapes

The model does not switch to a different mathematical block when it begins generating tokens. What changes is the shape of the work and what state can be reused.

| | Training/full sequence | Prompt prefill | Incremental decode |
|---|---|---|---|
| Query positions | many | many | usually one/few new positions |
| Past K/V | computed in pass | build cache | read from cache |
| MLP positions | many in parallel | many in parallel | new position(s) |
| Sequence dependency | causal mask for LM | causal mask | history already cached |

During decode, the newest token still passes through every Transformer block. At each attention sublayer, its query attends to cached K/V history. The MLP remains position-wise and only needs to process the new token state for that decode step.

**Mental model.** KV caching changes how the attention branch reuses history. It does not skip the rest of the block for the newly generated token.

*Research anchors: [4], [22]*

# 29 - 键值缓存（KV Cache） 属于 注意力机制（Attention） 状态

对应原文第 31 页；点击查看相邻原文对照页

29 - 键值缓存（KV Cache） 属于 注意力机制（Attention） 状态 跨解码步骤持续存在的内容 对于标准因果块，可重用的每个 词元（Token） 历史记录是 注意力机制（Attention） 键（Key）/值（Value） 状态（或特定于架构的压缩等效项），而不是整个块作为通用的“记忆”。在解码步骤中，层为新位置计算新的 查询（Query）/键（Key）/值（Value）。新的 查询（Query） 与早期位置缓存的 键（Keys） 和 值（Values） 进行交互。附加新的 K/V 以供后续步骤使用。 MLP 没有类似的序列缓存，因为它是位置式的：一旦当前的 词元（Token） 离开 MLP，该分支就不需要在未来的解码步骤中重新读取其早期的中间 激活（Activations）。

MLA 等架构确切地改变了缓存的内容，但可重用状态仍然是 注意力机制（Attention） 机制的一部分。[14]

重要。 键值缓存（KV Cache） 内存随着缓存的序列状态而增长。剩余流宽度和 MLP 宽度是单独的资源问题。

研究锚点：[14]、[22] 理解 Transformer 块（Transformer Block）

# 29 - THE KV CACHE BELONGS TO ATTENTION STATE

## What is persistent across decode steps

For a standard causal block, the reusable per-token history is the attention key/value state (or an architecture-specific compressed equivalent), not the output of the entire block as a generic "memory."

At decode step $t$, a layer computes a new query/key/value for the new position. The new query interacts with cached keys and values from earlier positions. The new K/V are appended for use by later steps. The MLP has no analogous sequence cache because it is position-wise: once the current token leaves the MLP, that branch has no need to reread its earlier intermediate activations during future decode steps.

Architectures such as MLA change exactly what is cached, but the reusable state is still part of the attention mechanism. [14]

**Important.** KV cache memory grows with cached sequence state. Residual-stream width and MLP width are separate resource questions.

*Research anchors: [14], [22]*

# 30 - 块级参数核算

对应原文第 32 页；点击查看相邻原文对照页

30 - 块级参数核算 注意力机制（Attention） 和 MLP 公式揭示了架构权衡 忽略偏差和规范，标准 MHA withHqdh =d 使用大约

使用 GQA，Q 和输出 矩阵（Matrices） 大致保持为 d2，而 K 和 V 仅投影到 Hkvdh：

福特 = 4096，Hq = 32，Hkv = 8，dh = 128：

对于该说明性配置，这些 注意力机制（Attention） 投影参数减少了 37.5%。

密集的两个 矩阵（Matrix）4d FFN 将包含 8d2 个 矩阵（Matrix） 参数，而参数匹配的 SwiGLU 门控激活（SwiGLU） 也接近 8d2。这些公式解释了为什么密集的 LLM 块通常将更多的 矩阵（Matrix） 参数分配给 MLP，而不是 注意力机制（Attention）。

重要：参数计数不是运行时的。序列长度、内核（Kernel） 效率、KV 流量、批处理（Batching） 和硬件可以使 注意力机制（Attention） 或 MLP 主导不同的执行机制。

研究锚点：[1]、[7]、[11] 了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 30 - BLOCK-LEVEL PARAMETER ACCOUNTING

## Attention and MLP formulas expose architecture trade-offs

Ignoring biases and norms, standard MHA with $H_q d_h = d$ uses approximately

$$P_{\text{MHA}} = 4d^2.$$

With GQA, the Q and output matrices remain roughly $d^2$ each, while K and V project only to $H_{kv} d_h$:

$$P_{\text{GQA}} \approx 2d^2 + 2d(H_{kv} d_h).$$

For $d = 4096$, $H_q = 32$, $H_{kv} = 8$, $d_h = 128$:

$$P_{\text{MHA}} = 67{,}108{,}864,$$

$$P_{\text{GQA}} = 41{,}943{,}040.$$

That is a 37.5% reduction in these attention-projection parameters for this illustrative configuration.

A dense two-matrix $4d$ FFN would contain $8d^2$ matrix parameters, while a parameter-matched SwiGLU is also near $8d^2$. These formulas explain why dense LLM blocks often devote more matrix parameters to the MLP than to attention.

**Important.** Parameter count is not runtime. Sequence length, kernel efficiency, KV traffic, batching, and hardware can make attention or the MLP dominate different execution regimes.

*Research anchors: [1], [7], [11]*

# 31 - 计算直觉：注意力机制（Attention） VS MLP

对应原文第 33 页；点击查看相邻原文对照页

31 - 计算直觉：注意力机制（Attention） VS MLP 序列混合和特征混合比例不同 对于序列长度 T 上的密集 自注意力（Self-attention），QK 和 AV 产品包含序列长度的二次项，粗略地说

当 头维度（Head Dimensions） 聚合时。 线性投影（Linear Projections） 贡献项大致成比例

密集 MLP 独立应用于每个位置，因此其占主导地位的 矩阵（Matrix） 乘法大致为 O(Tdm )

这意味着相对瓶颈取决于形状体系。在适度的上下文长度和较大的 模型（Model） 宽度下，MLP/投影 GEMM 可能是工作的主要部分。在长密集上下文中，注意力机制（Attention） 的成对序列项快速增长。在 one-词元（Token） 解码期间，注意力机制（Attention） 面临不断增长的 KV 读取，而 MLP 继续仅处理当前的 词元（Token）。

理解框架（Mental Model）。不要问“注意力机制（Attention） 还是 MLP 更贵？”无需指定序列长度、批次、宽度、架构、阶段、精度（Precision） 和硬件。

研究锚点：[1]、[22] 理解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 31 - COMPUTE INTUITION: ATTENTION VS MLP

### Sequence mixing and feature mixing scale differently

For dense self-attention over sequence length $T$, the QK and AV products contain a quadratic term in sequence length, roughly

$$O(T^2 d)$$

when head dimensions are aggregated. Linear projections contribute terms roughly proportional to $Td^2$.

A dense MLP applies independently to each position, so its dominant matrix multiplications scale roughly as

$$O(Tdm)$$

for a two-matrix FFN (with a constant-factor change for gated variants).

This means the relative bottleneck depends on the shape regime. At modest context length and large model width, MLP/projection GEMMs can be a major share of work. At long dense contexts, attention's pairwise sequence term grows rapidly. During one-token decoding, attention faces growing KV reads while the MLP continues processing only the current token.

**Mental model.** Do not ask "is attention or the MLP more expensive?" without specifying sequence length, batch, width, architecture, phase, precision, and hardware.

*Research anchors: [1], [22]*

# 32 - 激活（Activation） 内存和 重计算（Recomputation）

对应原文第 34 页：点击查看相邻原文对照页

32 - 激活（Activation） 内存和 重计算（Recomputation） 训练（Training） 存储多个参数 在 训练（Training） 期间，反向传播需要来自 前向传播（Forward Pass） 的信息。因此，除了权重、梯度（Gradient） 和优化器状态内存之外，Transformer 模型（Transformer） 块还会消耗大量的 激活（Activation） 内存。

简单的实现可以保留归一化输入、Q/K/V 投影、注意力机制（Attention） 中间体、MLP 激活（Activations） 以及向后 内核（Kernels） 所需的剩余状态。 融合内核（Fused Kernels） 和 重计算（Recomputation） 策略可以准确更改存储的内容。

激活（Activation） 检查点故意保存较少的中间 激活（Activations） 并在向后期间重新计算 前向传播（Forward Pass） 的部分，以额外的计算换取较低的 激活（Activation） 内存。这是围绕该块的 训练（Training） 系统策略，而不是对该块所代表的数学函数的更改。

重要：“块内存”是含糊不清的。参数内存、训练（Training） 激活（Activations）、优化器状态、临时 内核（Kernel） 工作区和 推理（Inference） 键值缓存（KV Cache） 是不同的数量。

研究锚：[25]

了解 Transformer 块（Transformer Block）

# 32 - ACTIVATION MEMORY AND RECOMPUTATION

## Training stores more than parameters

During training, backpropagation needs information from the forward pass. Transformer blocks can therefore consume substantial activation memory in addition to weight, gradient, and optimizer-state memory.

A naive implementation may retain normalized inputs, Q/K/V projections, attention intermediates, MLP activations, and residual states needed by backward kernels. Fused kernels and recomputation strategies can change exactly what is stored.

Activation checkpointing deliberately saves fewer intermediate activations and recomputes parts of the forward pass during backward, trading extra computation for lower activation memory. That is a training-system strategy around the block, not a change to the mathematical function the block represents.

**Important.** "Block memory" is ambiguous. Parameter memory, training activations, optimizer state, temporary kernel workspace, and inference KV cache are different quantities.

*Research anchors: [25]*

# 33 - 随机失活（Dropout）、偏见和其他小选择

对应原文第 35 页；点击查看相邻原文对照页

33 - 随机失活（Dropout）、偏差和其他小选择 配置细节并不通用 Transformer 模型（Transformer） 法则两种实现可以共享相同的高级框图，但在许多细节上有所不同：

- 线性投影（Linear Projections） 是否包含偏差项；

- 在何处应用随机失活（Dropout） 以及它在给定的模型（Model） 中是否为零；

- 归一化（Normalization） epsilon；

- 嵌入向量（Embeddings）/输出权重是否绑定；

- 精确的旋转尺寸和缩放（Scaling）；

- 注意力机制（Attention） 是否使用本地窗口、软上限或额外的归一化（Normalization）；

- 激活（Activation） 和门控MLP 约定；

- 张量（Tensor） 并联或融合内核（Kernel） 布局。

这些选择可能会影响检查点兼容性、参数计数、数值行为和 内核（Kernel） 选择。它们应该从 模型（Model） 的论文/配置/代码中读取，而不是从单词“Transformer 模型（Transformer）”中推断出来。Gemma 2 的 逻辑值（归一化前的分数）（Logit） 软上限和 Qwen3 的 QK-Norm 是架构细节的示例，这些细节很容易从通用图中省略，但对精确实现很重要。 [16,17]

研究锚点：[16]、[17] 理解 Transformer 块（Transformer Block）

# 33 - DROPOUT, BIASES, AND OTHER SMALL CHOICES

## Configuration details are not universal Transformer laws

Two implementations can share the same high-level block diagram while differing in many details:

- whether linear projections include bias terms;
- where dropout is applied and whether it is zero in a given model;
- the normalization epsilon;
- whether embeddings/output weights are tied;
- exact rotary dimensions and scaling;
- whether attention uses local windows, soft-capping, or extra normalization;
- activation and gated-MLP convention;
- tensor parallel or fused-kernel layout.

These choices can affect checkpoint compatibility, parameter counts, numerical behavior, and kernel selection. They should be read from the model's paper/configuration/code rather than inferred from the word "Transformer."

Gemma 2's logit soft-capping and Qwen3's QK-Norm are examples of architecture details that are easy to omit from a generic diagram but matter to an exact implementation. [16,17]

*Research anchors: [16], [17]*

# 34 - 透明的 Pytorch 块

对应原文第 36 页；点击查看相邻原文对照页

34 - 透明 PYTORCH 块 教学预 均方根归一化（RMSNorm） 注意力机制（Attention） + SwiGLU 门控激活（SwiGLU） 以下可执行草图公开了操作顺序。它使用普通的 MHA，并有意忽略生产问题，例如 GQA、RoPE、KV 缓存、融合内核（Fused Kernels）、张量（Tensor） 并行性、随机失活（Dropout） 和 模型（Model） 特定的偏差。

类 CausalSelfAttention(nn.Module):

super().__init__() 断言 d % n_头（Heads） == 0 self.h, self.dh = n_头（Heads）, d // n_头（Heads） self.qkv = nn.Linear(d, 3 * d, 偏差=False) self.out = nn.Linear(d, d, 偏差=False)

q, k, v = self.qkv(x).chunk(3, 暗淡=-1)

q, k, v = 地图(头（Heads）, (q, k, v)) y = F.scaled_dot_product_注意力机制（Attention）(q, k, v, is_causal=True) y = y.transpose(1, 2).reshape(B, T, D)

类块（nn.Module）：

super().__init__() self.norm1, self.norm2 = nn.均方根归一化（RMSNorm）(d), nn.均方根归一化（RMSNorm）(d) self.attn = CausalSelfAttention(d, n_头（Heads）) self.gate = nn.Linear(d, mlp_dim, 偏差=False) self.up = nn.Linear(d, mlp_dim, 偏差=False) self.down = nn.Linear(mlp_dim, d, 偏差=False)

x = x + self.attn(self.norm1(x)) h = self.norm2(x) h = F.silu(self.gate(h)) * self.up(h)

这与串行 前置归一化（Pre-norm） 残差拓扑和三投影 SwiGLU 门控激活（SwiGLU） 结构相匹配。真实的 模型（Model） 必须与其 精确注意力（Exact Attention） 头布局、RoPE 规则、范数 epsilon、MLP 维度、缓存语义和检查点参数名称匹配。

重要：教学代码不是 LLaMA、Qwen、Gemma 或其他 模型（Model） 的直接复制。精确的架构是一个兼容性契约。

研究锚点：[6]、[7]、[8]、[17]、[20]、[22] 了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 34 - A TRANSPARENT PYTORCH BLOCK

## Pedagogical Pre-RMSNorm attention + SwiGLU

The following executable sketch exposes the order of operations. It uses ordinary MHA and intentionally omits production concerns such as GQA, RoPE, KV caching, fused kernels, tensor parallelism, dropout, and model-specific biases.

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        assert d % n_heads == 0
        self.h, self.dh = n_heads, d // n_heads
        self.qkv = nn.Linear(d, 3 * d, bias=False)
        self.out = nn.Linear(d, d, bias=False)

    def forward(self, x):
        B, T, D = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        def heads(z):
            return z.view(B, T, self.h, self.dh).transpose(1, 2)
        q, k, v = map(heads, (q, k, v))
        y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        y = y.transpose(1, 2).reshape(B, T, D)
        return self.out(y)

class Block(nn.Module):
    def __init__(self, d, n_heads, mlp_dim):
        super().__init__()
        self.norm1, self.norm2 = nn.RMSNorm(d), nn.RMSNorm(d)
        self.attn = CausalSelfAttention(d, n_heads)
        self.gate = nn.Linear(d, mlp_dim, bias=False)
        self.up = nn.Linear(d, mlp_dim, bias=False)
        self.down = nn.Linear(mlp_dim, d, bias=False)

    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        h = self.norm2(x)
        h = F.silu(self.gate(h)) * self.up(h)
        return x + self.down(h)
```

This matches the serial Pre-Norm residual topology and the three-projection SwiGLU structure. A real model must match its exact attention head layout, RoPE rule, norm epsilon, MLP dimensions, cache semantics, and checkpoint parameter names.

**Important.** Pedagogical code is not a drop-in reproduction of LLaMA, Qwen, Gemma, or another model. Exact architecture is a compatibility contract.

*Research anchors: [6], [7], [8], [17], [20], [22]*

# 35 - 常见的实施错误

对应原文第 37 页；点击查看相邻原文对照页

35 - 常见实施错误 大多数块错误是顺序、形状或契约错误

1. 将 Post-LN 方程用于 Pre-LN 检查点。相同的模块，不同的功能。

2.归一化错误的张量（Tensor）.Block范数、QK-Norm和post-sublayer范数是不同的站点。

3. 忘记注意力机制（Attention） 输出投影（Output Projection）。头连接不是完整分支。

4. 默认情况下将 RoPE 应用于 残差流（Residual Stream） 或 V。标准 LLaMA 式 RoPE 目标为 Q/K。

5. 假设Hq=Hkv。 GQA/MQA 打破了这种平等。

6. 对 SwiGLU 门控激活（SwiGLU）.门控（Gating） 使用两个 矩阵（Matrix） FFN 参数公式添加第三个大投影。

7. 添加宽度不匹配的张量。每个剩余分支必须返回 tod。

8. 将因果屏蔽视为块级操作。它属于 注意力机制（Attention） 连接性。

9. 复制通用 MLP 宽度。扩展比率是特定于体系结构的。

10. 将工作的 张量形状（Tensor Shape） 与检查点兼容性等同起来。Epsilon、偏差、捆绑、排序和约定仍然很重要。

研究锚点：[1]、[8]、[11]、[12]、[16]、[17] 理解 Transformer 块（Transformer Block）

# 35 - COMMON IMPLEMENTATION MISTAKES

## Most block bugs are ordering, shape, or contract bugs

1. **Using Post-LN equations for a Pre-LN checkpoint.** Same modules, different function.
2. **Normalizing the wrong tensor.** Block norm, QK-Norm, and post-sublayer norm are different sites.
3. **Forgetting the attention output projection.** Head concatenation is not the full branch.
4. **Applying RoPE to the residual stream or V by default.** Standard LLaMA-style RoPE targets Q/K.
5. **Assuming $H_q = H_{kv}$.** GQA/MQA break that equality.
6. **Using a two-matrix FFN parameter formula for SwiGLU.** Gating adds a third large projection.
7. **Adding tensors with mismatched widths.** Every residual branch must return to $d$.
8. **Treating causal masking as a block-level operation.** It belongs inside attention connectivity.
9. **Copying a generic MLP width.** Expansion ratios are architecture-specific.
10. **Equating a working tensor shape with checkpoint compatibility.** Epsilon, bias, tying, ordering, and conventions still matter.

*Research anchors: [1], [8], [11], [12], [16], [17]*

# 36 - 常见的误解

对应原文第 38 页；点击查看相邻原文对照页

36 - 常见错误 创建错误 理解框架（Mental Model） 的快捷方式 错误概念纠正“注意力机制（Attention） 输出取代了 词元（Token） 状态。”它通常作为剩余更新添加。

“MLP 混合了 词元（Tokens）。”它按位置行事；上下文可能已经存在于每个状态中。

“均方根归一化（RMSNorm） 是 层归一化（LayerNorm），没有偏见。”均方根归一化（RMSNorm） 还省略了均值减法并使用不同的尺度统计量。

“LN 之前和 LN 后是等效的重新排序。”它们定义了不同的计算/梯度（Gradient） 路径。

“RoPE 修改 词元嵌入（Token Embeddings）。”标准 RoPE 在 注意力机制（Attention） 内旋转 Q/K。

“每个现代法学硕士都是 Pre-均方根归一化（RMSNorm） + SwiGLU 门控激活（SwiGLU）。” Gemma 2、PaLM、Qwen3 等说明了重要的变体。

“GQA 改变了整个区块。”它改变了注意力机制（Attention）的Q/K/V头结构。

“MoE 表示不同的 Transformer 模型（Transformer） 骨架。”它通常用路由的 专家（Experts） 替换 前馈（Feed-forward） 分支。

“键值缓存（KV Cache） 存储整个区块状态。”它存储 注意力机制（Attention） K/V 或特定于体系结构的等效项。

“注意力机制（Attention） 始终主导计算。”瓶颈取决于形状、阶段、架构和硬件。

研究锚点：[1]、[6]、[8]、[10]、[11]、[14]、[16]、[17] 了解 Transformer 块（Transformer Block）

# 36 - COMMON MISCONCEPTIONS

## Shortcuts that create the wrong mental model

| Misconception | Correction |
|---|---|
| “The attention output replaces the token state.” | It is usually added as a residual update. |
| “The MLP mixes tokens.” | It acts position-wise; context can already be present in each state. |
| “RMSNorm is LayerNorm without bias.” | RMSNorm also omits mean subtraction and uses a different scale statistic. |
| “Pre-LN and Post-LN are equivalent reorderings.” | They define different computation/gradient paths. |
| “RoPE modifies token embeddings.” | Standard RoPE rotates Q/K inside attention. |
| “Every modern LLM is Pre-RMSNorm + SwiGLU.” | Gemma 2, PaLM, Qwen3, and others illustrate important variants. |
| “GQA changes the whole block.” | It changes attention’s Q/K/V head structure. |
| “MoE means a different Transformer skeleton.” | It commonly replaces the feed-forward branch with routed experts. |
| “The KV cache stores the whole block state.” | It stores attention K/V or an architecture-specific equivalent. |
| “Attention always dominates compute.” | The bottleneck depends on shape, phase, architecture, and hardware. |

*Research anchors: [1], [6], [8], [10], [11], [14], [16], [17]*

# 37 - 完整的 理解框架（Mental Model）

对应原文第 39 页；点击查看相邻原文对照页

37 - 完整的 理解框架（Mental Model） 一条链中的 Transformer 块（Transformer Block） 现代 Transformer 块（Transformer Block） 最容易记住的是九个相互关联的想法。

1. 携带剩余状态。每个 词元（Token） 都有一个宽度为 d 的 向量（Vector），该状态在堆栈中持续存在。

2. 在定义的站点进行标准化。层归一化（LayerNorm）/均方根归一化（RMSNorm） 放置是架构的一部分。

3. 将 词元（Tokens） 与 注意力机制（Attention） 混合。Q/K/V 投影、位置处理、掩蔽和头布局决定位置如何交互。

4. 项目回溯。 注意力机制（Attention） 分支必须返回到剩余宽度。

5.添加更新。剩余添加保留了旧的 表示（Representation），同时纳入了 注意力机制（Attention） 的贡献。

6. 使用 MLP 变换特征。每个 词元（Token） 都经过扩展、非线性选通/激活，并独立投影回来。

7. 再次添加。MLP 贡献另一个剩余更新。

8. 对特定于架构的变体重复。GQA、QK-Norm、GeGLU/SwiGLU 门控激活（SwiGLU）、局部注意力（Local Attention）、后置归一化（Post-norm）、MoE 和其他选择会更改内部结构，而不会擦除 核心（Core） 逻辑。

9. 按phase.训练（Training）、预填充（Prefill）解释执行，并解码在不同形状和重用模式下运行相同的学习块。

理解框架（Mental Model）。持久抽象不是“注意力机制（Attention） 后跟 MLP”。它是由 词元（Token） 混合和特征混合计算重复更新的残余状态，其 归一化（Normalization）、路由（Routing） 和 注意力机制（Attention） 详细信息是特定于体系结构的。

研究锚点：[1]、[4]、[6]、[7]、[8]、[10]、[16]、[17] 了解 Transformer 块（Transformer Block）

# 37 - THE COMPLETE MENTAL MODEL

## The Transformer block in one chain

A modern Transformer block is easiest to remember as nine linked ideas.

1. **Carry a residual state.** Each token has a width-$d$ vector that persists through the stack.
2. **Normalize at defined sites.** LayerNorm/RMSNorm placement is part of the architecture.
3. **Mix tokens with attention.** Q/K/V projections, position treatment, masking, and head layout determine how positions interact.
4. **Project back to $d$.** The attention branch must return to residual width.
5. **Add the update.** Residual addition preserves the old representation while incorporating attention's contribution.
6. **Transform features with the MLP.** Each token is expanded, nonlinearly gated/activated, and projected back independently.
7. **Add again.** The MLP contributes another residual update.
8. **Repeat with architecture-specific variants.** GQA, QK-Norm, GeGLU/SwiGLU, local attention, post-norm, MoE, and other choices alter internals without erasing the core logic.
9. **Interpret execution by phase.** Training, prefill, and decode run the same learned block under different shapes and reuse patterns.

**Mental model.** The durable abstraction is not "attention followed by an MLP." It is a residual state repeatedly updated by token-mixing and feature-mixing computations whose normalization, routing, and attention details are architecture-specific.

*Research anchors: [1], [4], [6], [7], [8], [10], [16], [17]*

# 38 - 紧凑架构比较

对应原文第 40 页；点击查看相邻原文对照页

38 - 紧凑架构比较 代表性块，不是通用进展 模型（Model）/系列规范放置 MLP 位置/注意力机制（Attention） 有用的课程 Transformer 模型（Transformer） (2017) Post-LN ReLU，4d 正弦 + MHA 原始参考块 GPT-2 Pre-LN + 最终规范 GELU 式因果 MHA pre-归一化（Normalization） 移入子块输入 PaLM Pre-LN，并行分支 SwiGLU 门控激活（SwiGLU） MQA 不需要串行排序 LLaMA Pre-均方根归一化（RMSNorm） SwiGLU 门控激活（SwiGLU） RoPE + MHA 通用现代密集配方 Gemma 2 pre + post RM-SNorm GeGLU GQA，局部/全局现代规范放置仍然不同 Qwen3 Pre-均方根归一化（RMSNorm） + QK-Norm SwiGLU 门控激活（SwiGLU） RoPE + GQA 内部 注意力机制（Attention） 规范可以与块规范共存 该表是有意选择的。它说明了为什么短语“Transformer 块（Transformer Block）”应该触发一个清单而不是单个记忆绘图：范数放置、分支排序、注意力头（Attention Head） 结构、位置方法、MLP 门控（Gating） 和内部 注意力机制（Attention） 修改都可以更改。

重要： 模型（Model） 系列在不同版本和参数范围内不断发展。为了准确再现，请使用相应的技术报告和发布的配置，而不是此比较表。

研究锚点：[1]、[4]、[8]、[10]、[16]、[17] 了解 Transformer 块（Transformer Block）

# 38 - A COMPACT ARCHITECTURE COMPARISON

## Representative blocks, not a universal progression

| Model/family | Norm placement | MLP | Position/attention | Useful lesson |
|---|---|---|---|---|
| Transformer (2017) | Post-LN | ReLU, $4d$ | sinusoidal + MHA | original reference block |
| GPT-2 | Pre-LN + final norm | GELU-style | causal MHA | pre-normalization moved into sub-block input |
| PaLM | Pre-LN, parallel branches | SwiGLU | MQA | serial ordering is not required |
| LLaMA | Pre-RMSNorm | SwiGLU | RoPE + MHA | common modern dense recipe |
| Gemma 2 | pre + post RMSNorm | GeGLU | GQA, local/global | modern norm placement still varies |
| Qwen3 | Pre-RMSNorm + QK-Norm | SwiGLU | RoPE + GQA | internal attention norms can coexist with block norm |

This table is intentionally selective. It shows why the phrase “Transformer block” should trigger a checklist rather than a single memorized drawing: norm placement, branch ordering, attention head structure, positional method, MLP gating, and internal attention modifications can all change.

**Important.** Model families evolve across versions and parameter scales. For exact reproduction, use the corresponding technical report and released configuration rather than this comparison table.

*Research anchors: [1], [4], [8], [10], [16], [17]*

# 39 - 参考文献和扩展阅读

对应原文第 41 页；点击查看相邻原文对照页

39 - 参考文献和扩展阅读

1. A. Vaswani 等人。注意力机制（Attention） 就是您所需要的。 NeurIPS 2017。arXiv：1706.03762。

2. 何坤，张晓，任书，孙杰。用于图像识别的深度残差学习。 CVPR 2016。

arXiv：1512.03385。

3. J. L. Ba、J. R. Kiros、G. E. Hinton.层归一化（Layer Normalization）。 arXiv：1607.06450，2016。

4. A. Radford et al.Language 模型（Models） 是无监督多任务学习者。 OpenAI技术报告，

5. R. Xiong 等人关于 Transformer 模型（Transformer） 架构中的 层归一化（Layer Normalization）。 ICML 2020。arXiv：2002.04745。

6.B.Zhang，R.Sennrich.均方根层归一化（Layer Normalization）。 NeurIPS 2019。arXiv：1910.07467。

7.N.沙泽尔。 GLU 变体改进了 Transformer 模型（Transformer）。 arXiv：2002.05202，2020。

8. H. Touvron 等人.LLaMA：开放高效的基础语言 模型（Models）。 arXiv：2302.13971，2023。

9. H. Touvron 等人。Llama 2：开放基础和微调聊天 模型（Models）。 arXiv：2307.09288，2023。

10. A. Chowdhery 等人。PaLM：缩放（Scaling） 语言建模与路径。 JMLR 2023 年 24 月。

11.J.安斯利等人。 GQA：训练（Training） 来自多头检查点的通用多 查询（Query） Transformer 模型（Transformer） 模型（Models）。 EMNLP 2023。arXiv：2305.13245。

12. J. Su et al.RoFormer：使用 旋转位置嵌入（Rotary Position Embedding） 增强 Transformer 模型（Transformer）。 arXiv:2104.09864,

13. T. Dao 等人FlashAttention：具有 IO 感知功能的快速且内存高效的 精确注意力（Exact Attention）。神经信息处理系统

2022.arXiv：2205.14135。

14. DeepSeek-AI 等人。DeepSeek-V2：一种强大、经济、高效的 混合专家（Mixture-of-Experts） 语言 模型（Model）。 arXiv：2405.04434，2024。

15. A. Jiang 等人。专家（Experts） 的混合。 arXiv：2401.04088，2024。

16. Gemma Team.Gemma 2：以实用的规模改进开放语言 模型（Models）。 arXiv:2408.00118,

17. A. Yang 等人的 Qwen3 技术报告。 arXiv：2505.09388，2025。

18. T. Brown 等人的语言 模型（Models） 是少样本学习者。 NeurIPS 2020。arXiv：2005.14165。

19.PyTorch 文档。火炬.nn.层归一化（LayerNorm）。当前文档。

20.PyTorch 文档。火炬.nn.均方根归一化（RMSNorm）。当前文档。

21. H. Wang 等人。DeepNet：缩放（Scaling） 转换器到 1,000 层。 arXiv：2203.00555，2022。

22.PyTorch 文档。 torch.nn.function.scaled_dot_product_注意力机制（Attention）。当前文档。

23. D. Hendrycks，K. Gimpel。高斯误差线性单元（GELU）。 arXiv：1606.08415，2016。

24. N. Shazeer 等人，超大神经网络：稀疏门控 混合专家（Mixture-of-Experts） 层。

ICLR 2017。arXiv：1701.06538。

25. T. Chen 等人。训练（Training） 具有次线性内存成本的深度网络。 arXiv：1604.06174，2016。

了解 Transformer 块（Transformer Block）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 39 - REFERENCES AND FURTHER READING


1. A. Vaswani et al. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762.
2. K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition.* CVPR 2016. arXiv:1512.03385.
3. J. L. Ba, J. R. Kiros, G. E. Hinton. *Layer Normalization.* arXiv:1607.06450, 2016.
4. A. Radford et al. *Language Models are Unsupervised Multitask Learners.* OpenAI technical report, 2019.
5. R. Xiong et al. *On Layer Normalization in the Transformer Architecture.* ICML 2020. arXiv:2002.04745.
6. B. Zhang, R. Sennrich. *Root Mean Square Layer Normalization.* NeurIPS 2019. arXiv:1910.07467.
7. N. Shazeer. *GLU Variants Improve Transformer.* arXiv:2002.05202, 2020.
8. H. Touvron et al. *LLaMA: Open and Efficient Foundation Language Models.* arXiv:2302.13971, 2023.
9. H. Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models.* arXiv:2307.09288, 2023.
10. A. Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways.* JMLR 24, 2023.
11. J. Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.* EMNLP 2023. arXiv:2305.13245.
12. J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* arXiv:2104.09864, 2021.
13. T. Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. arXiv:2205.14135.
14. DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* arXiv:2405.04434, 2024.
15. A. Jiang et al. *Mixtral of Experts.* arXiv:2401.04088, 2024.
16. Gemma Team. *Gemma 2: Improving Open Language Models at a Practical Size.* arXiv:2408.00118, 2024.
17. A. Yang et al. *Qwen3 Technical Report.* arXiv:2505.09388, 2025.
18. T. Brown et al. *Language Models are Few-Shot Learners.* NeurIPS 2020. arXiv:2005.14165.
19. PyTorch documentation. `torch.nn.LayerNorm`. Current documentation.
20. PyTorch documentation. `torch.nn.RMSNorm`. Current documentation.
21. H. Wang et al. *DeepNet: Scaling Transformers to 1,000 Layers.* arXiv:2203.00555, 2022.
22. PyTorch documentation. `torch.nn.functional.scaled_dot_product_attention`. Current documentation.
23. D. Hendrycks, K. Gimpel. *Gaussian Error Linear Units (GELUs).* arXiv:1606.08415, 2016.
24. N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* ICLR 2017. arXiv:1701.06538.
25. T. Chen et al. *Training Deep Nets with Sublinear Memory Cost.* arXiv:1604.06174, 2016.